Program Analysis for Quantitative-reachability Properties

Jiawen Liu
Advised by: Marco Gaboardi

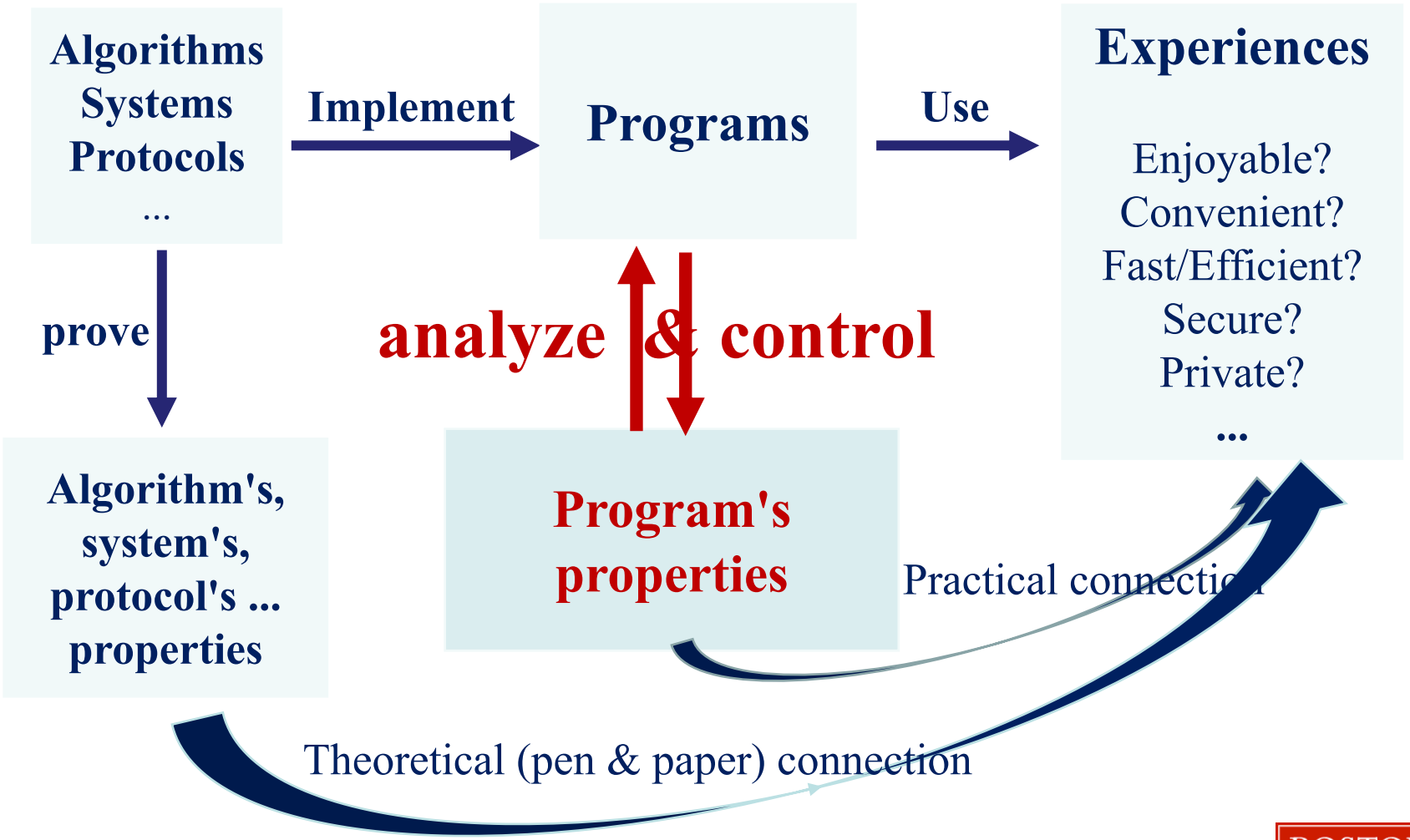
Boston University Department of Computer Science

BOSTON UNIVERSITY

Outline

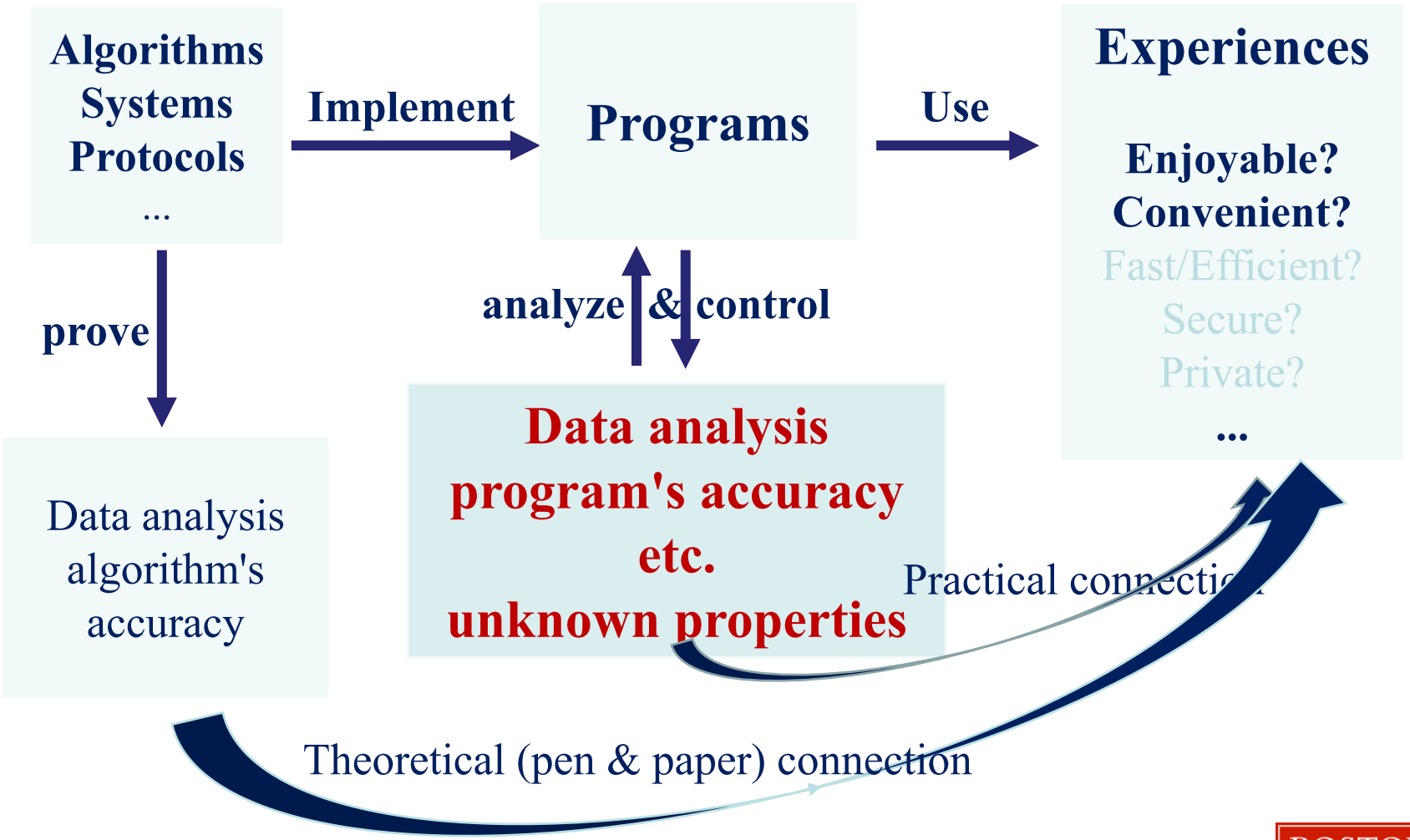
- Introduction
- Program Analysis for Adaptive Data Analysis
  - Introduction
  - QWhile Language
  - Adaptivity Formalization
  - Program Analysis Algorithm
- Path-sensitive Reachability-bound Analysis (In Submission)
  - Introduction
  - Language Model
  - Reachability-bound
  - Path-sensitive Reachability-bound Algorithm
- Future Works

Introduction – program analysis background



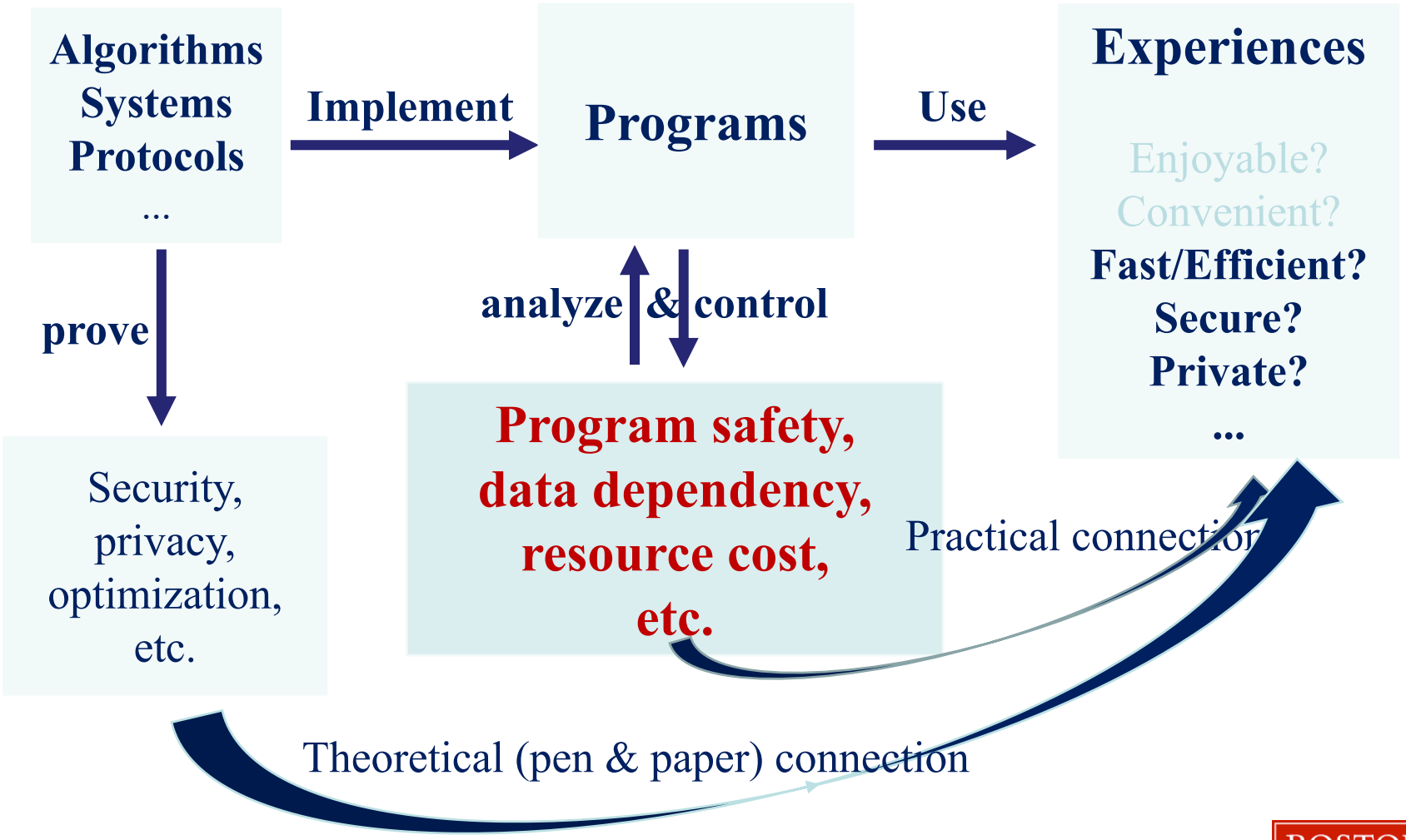
BOSTON UNIVERSITY

Introduction – program analysis background

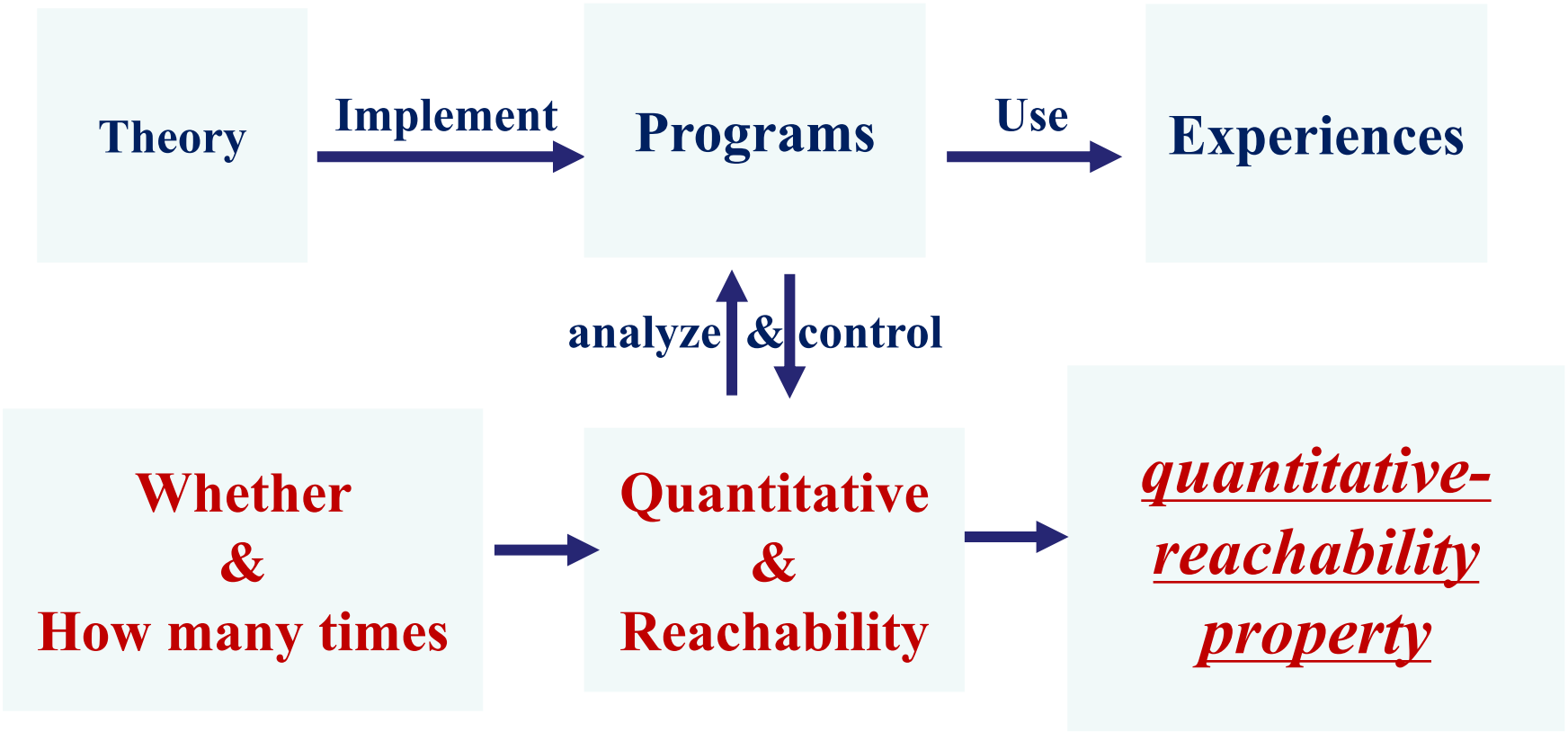


BOSTON UNIVERSITY

Introduction – program analysis background



Introduction – background



Introduction – challenge : property formalization challenge

quantitative-reachability properties

sub-properties

Variable/Data/Code reachability, commutative, liveness, etc

Quantitative & Reachability

Resource cost / Worst-case execution time / Complexity, etc

Introduction – *new methodology*

Semantics-based quantitative-reachability formalization

- Define sub-property "*quantitative*" & *"reachability*",
- ➔
- Define new *structure/model* carrying the two sub-properties,
- ➔
- Define the "*quantitative-reachability*" as a hyper-property over the model

Introduction – challenge : program analysis challenge

quantitative-reachability properties

sub-properties

Control-flow, data-flow, liveness, dependency analysis, etc.

Quantitative & Reachability

Resource cost, worst-case execution time, complexity estimation

reachability ***reachability ∞ quantity*** **overall quantity**

Introduction – new methodology

➔ ***Analyze the quantitative-reachability integrally:***

Analyze sub-properties "***quantitative***" & ***"reachability***"

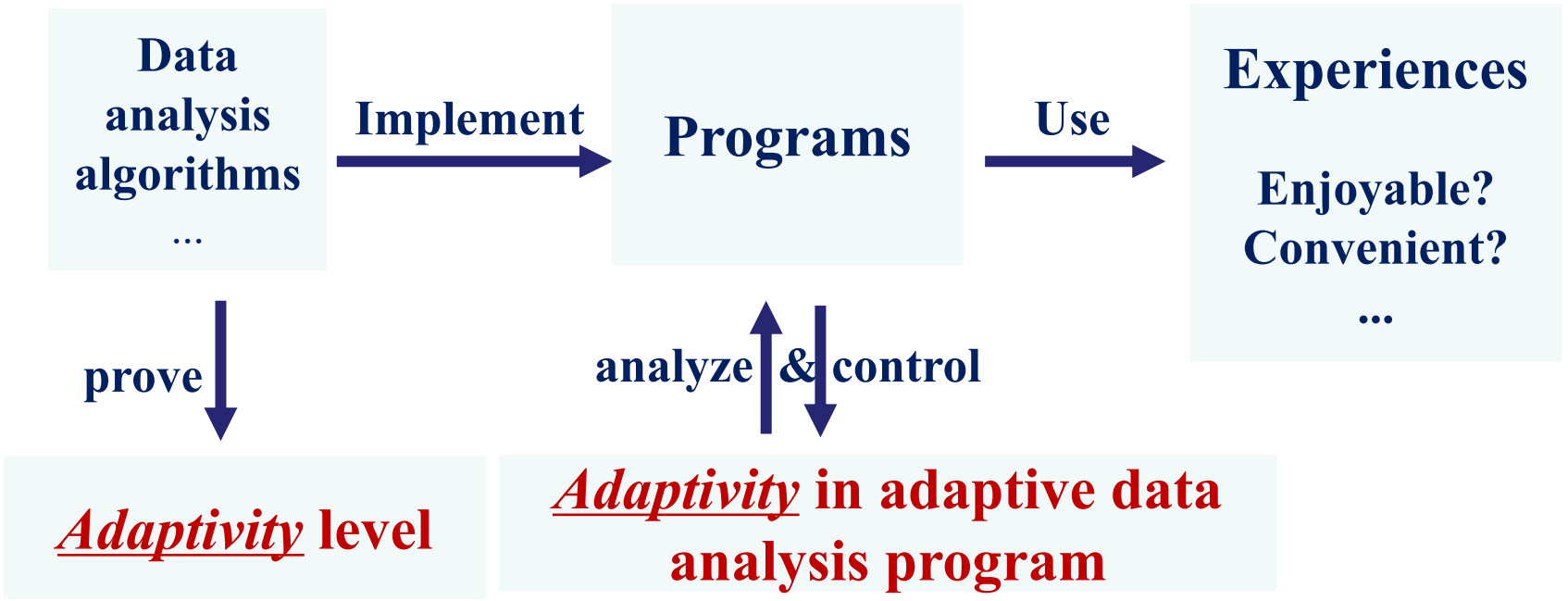
➔

Estimate the new ***structure/model***

➔

Analyze the hyper-property "***quantitative-reachability***"

Introduction – two quantitative-reachability properties



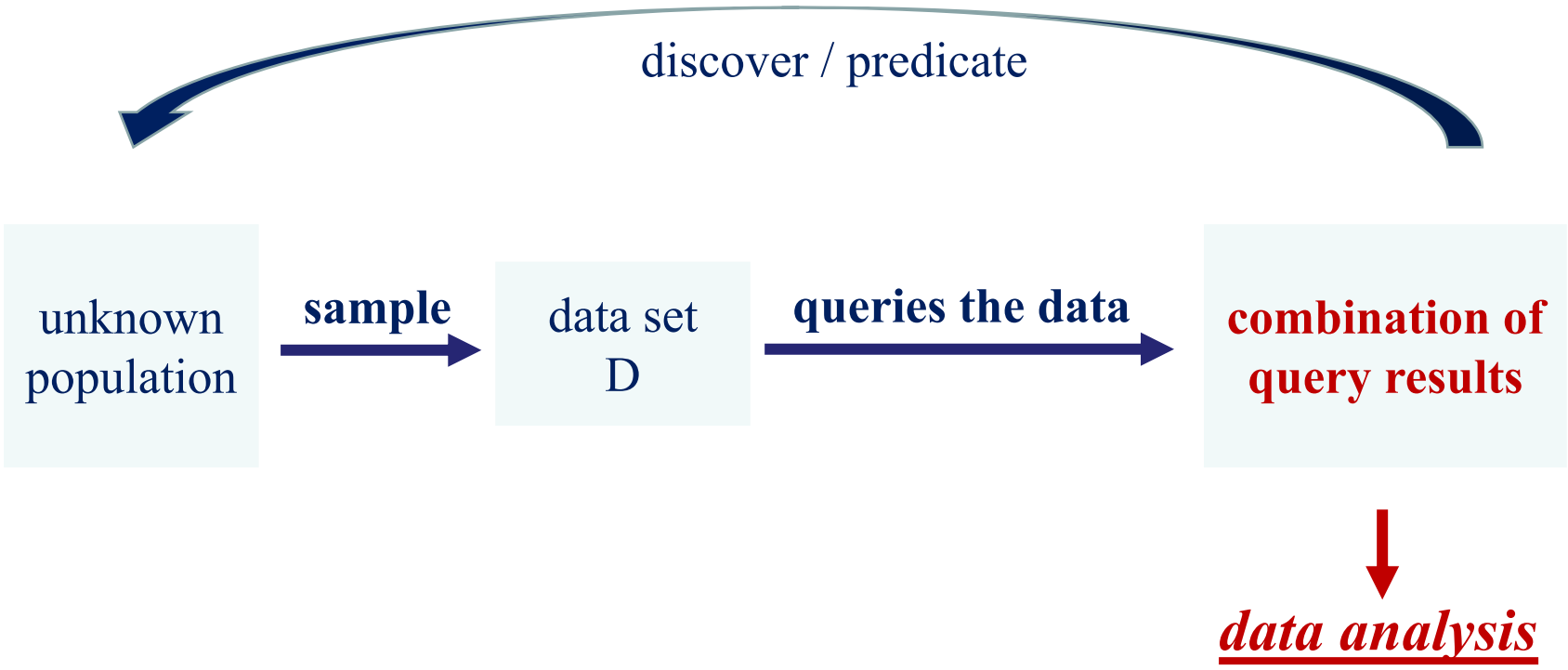
Introduction – two quantitative-reachability properties

The visiting times of each program location during the program execution

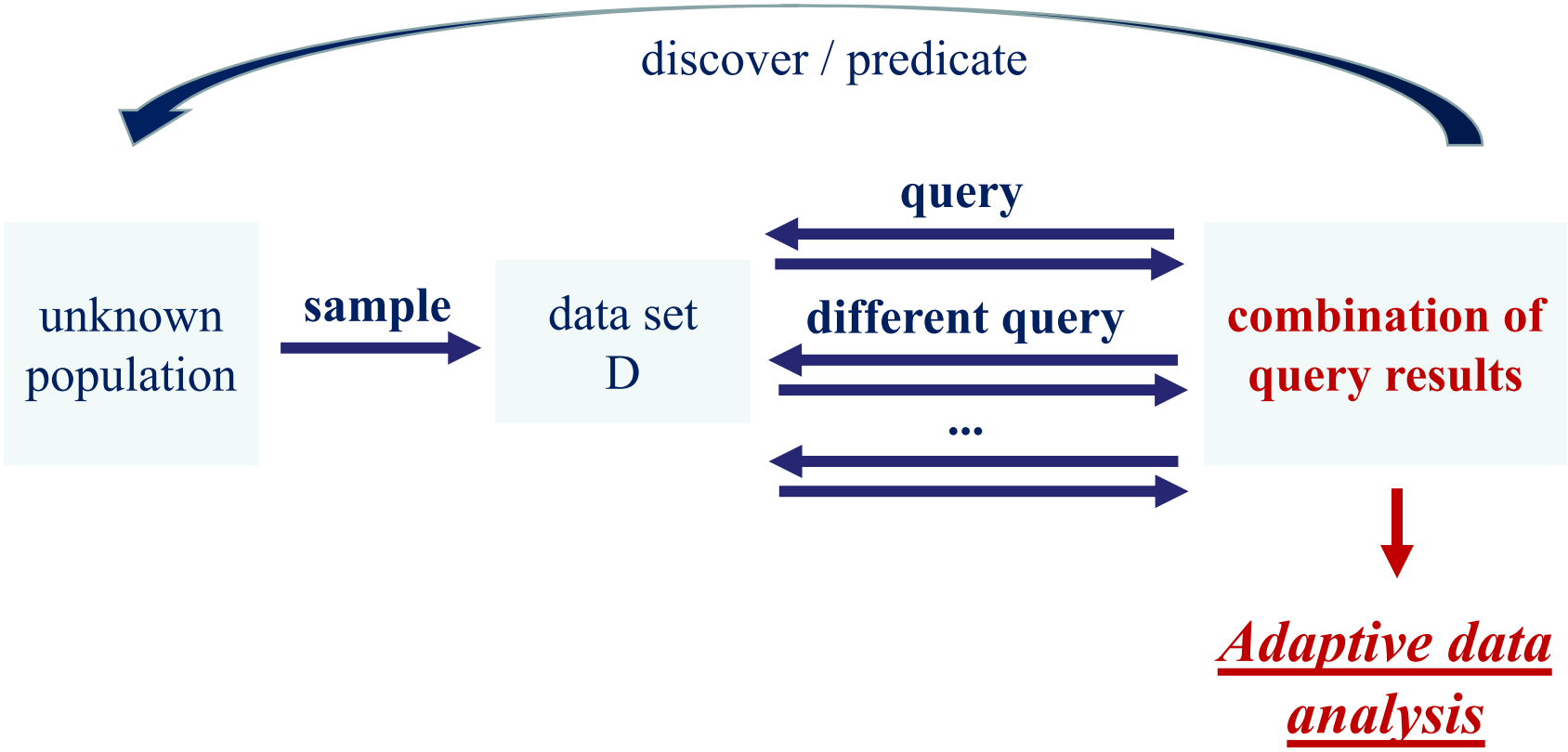
PART I :
Program Analysis for Adaptive Data Analysis

- Introduction:
  - Adaptivity
  - Motivation
- QWhile Language
- Adaptivity Definition
- Adaptivity Estimation Algorithm

Introduction – adaptive data analysis



Introduction – adaptive data analysis

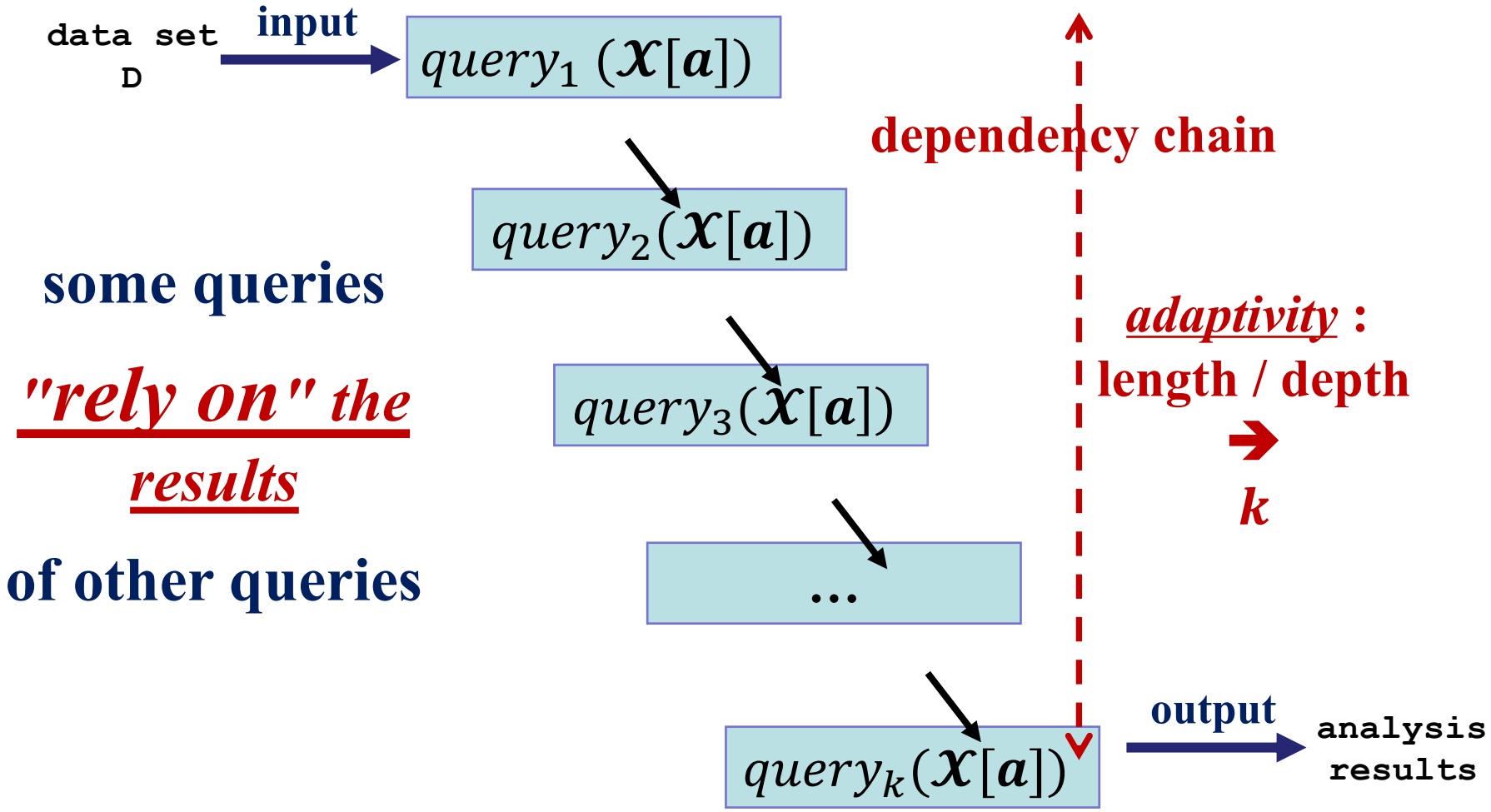


BOSTON UNIVERSITY

example program

```
multiRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do (
      [a ← query(𝒳[a])]3;
      [j ← j - 1 ]4
      )
```

Introduction – adaptivity in adaptive data analysis

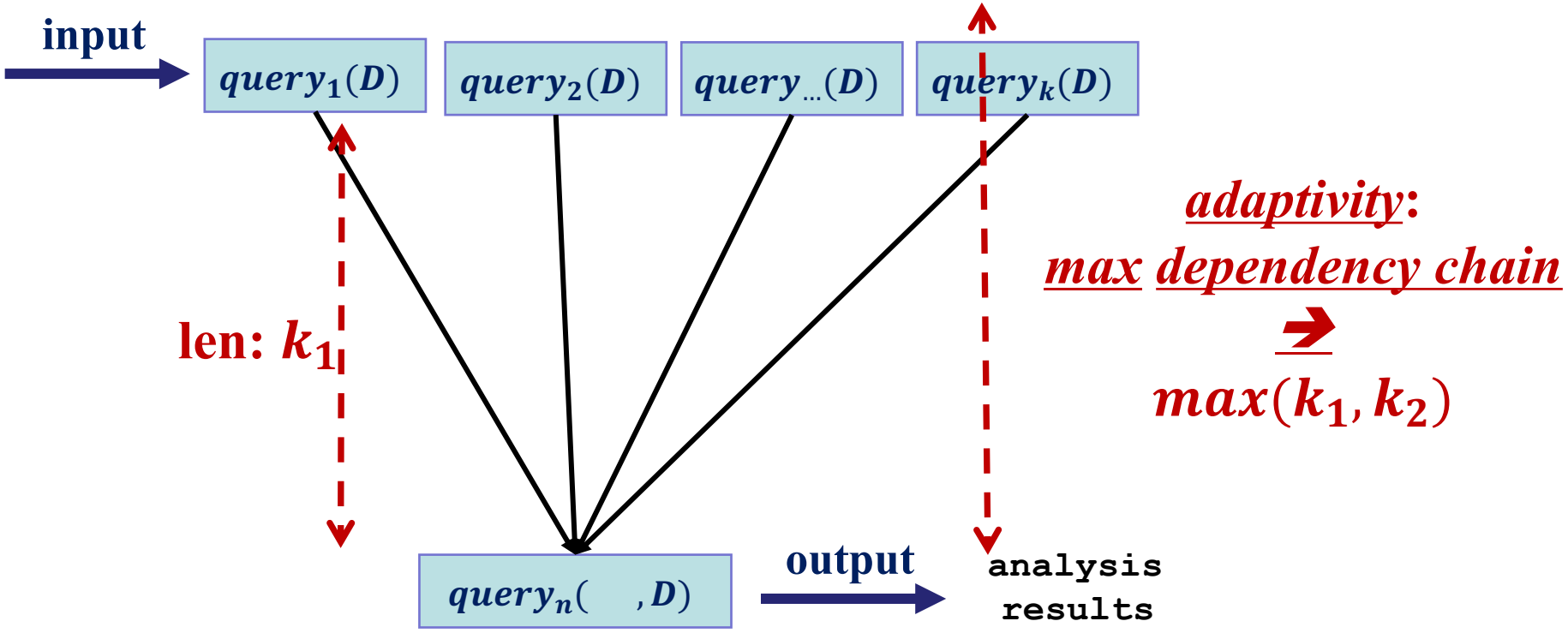


BOSTON UNIVERSITY

QWhile language – example program

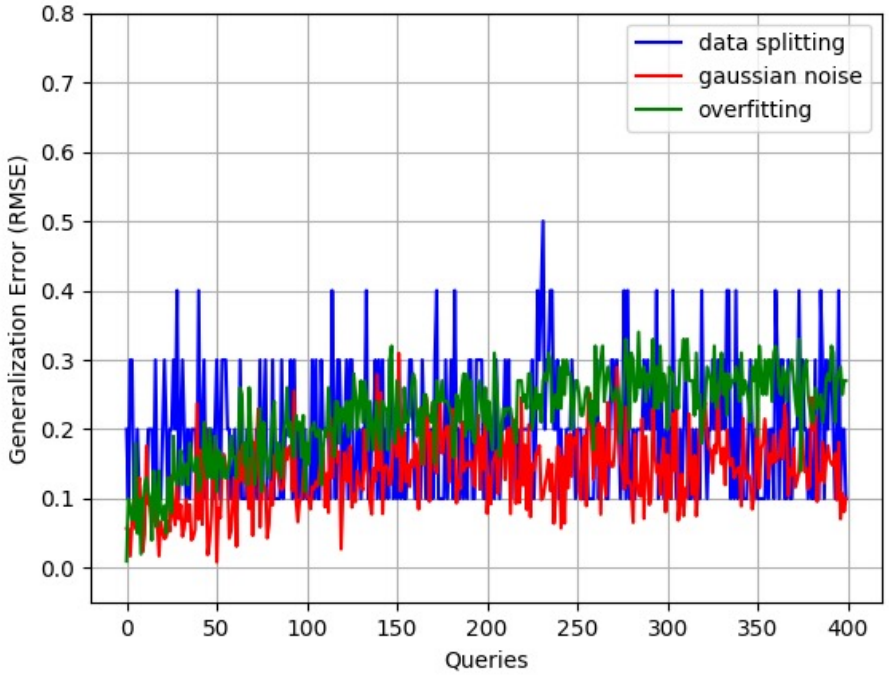
```
twoRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do (
      [x ← query(𝒳[j])]3;
      [j ← j - 1 ]4;
      [a ← a + x]5);
[l ← query(𝒳[k] * a)]6
```

Introduction – adaptivity in adaptive data analysis



the "*adaptivity*" in a data analysis propagates the error

Introduction – control the generalization error theoretically



use "*adaptivity*" to choose different mechanisms,

to reduce the generalization error

BOSTON UNIVERSITY

Introduction – Research challenges

- Challenge – I : program model for implementing an adaptive data analysis algorithm
- Challenge – II : how to define this quantitative-reachability property
- Challenge – III : how to estimate this quantitative-reachability property accurately

Contributions

- I : QWhile language
- II : Formal adaptivity definition
- III : Program analysis algorithm for adaptivity
- IV: Soundness proof
- V: Prototype implementation

Adaptivity Analysis Framework

QWhile language

Expression	e	::=	$v \mid a \mid b \mid [e, \ldots, e]$
Value	v	::=	$n \mid \texttt{true} \mid \texttt{false} \mid [] \mid [v, \ldots, v]$
Query Expression	ψ	::=	$\alpha \mid a \mid \psi \oplus_a \psi \mid \chi[a]$
Query Value	α	::=	$n \mid \chi[n] \mid \alpha \oplus_a \alpha \mid n \oplus_a \chi[n] \mid \chi[n] \oplus_a n$
Labeled Command	c	::=	$[x \leftarrow e]^l \mid [x \leftarrow \texttt{query}(\psi)]^l \mid \texttt{while}\ [b]^l\ \texttt{do}\ c$
			$\mid c; c \mid \texttt{if}\ ([b]^l, c, c) \mid [\texttt{skip}]^l$

QWhile language – ***query expression***

Query Expression	ψ	::=	$\alpha \mid a \mid \psi \oplus_a \psi \mid \chi[a]$
Query Value	α	::=	$n \mid \chi[n] \mid \alpha \oplus_a \alpha \mid n \oplus_a \chi[n] \mid \chi[n] \oplus_a n$

- χ[n] : access the values at ***n-th column*** of a ***hidden*** database
- query(χ[n]) : the ***average*** value of χ[n] over each row of the database

$$\mathtt{query}(\chi[3]+5) = \frac{1}{n}\sum_{i=0}^{n}\chi_i[3]+5$$

QWhile language – trace-based semantics

- ***Query command evaluation***

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \qquad \texttt{query}(\alpha) = v \qquad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \texttt{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\texttt{skip}]^l, \tau :: \epsilon \rangle} \textbf{ query}$$

- 1. Normalize the query expression: $\langle \tau,\psi \rangle \Downarrow q\ \alpha$.

The query expression ψ is first simplified to its normal form α

$$\alpha \quad ::= \quad n \mid \chi[n] \mid \alpha \oplus_a \alpha \mid n \oplus_a \chi[n] \mid \chi[n] \oplus_a n$$

$$\frac{\langle \tau, a \rangle \Downarrow_a n}{\langle \tau, a \rangle \Downarrow_q n} \qquad \frac{\langle \tau, \psi_1 \rangle \Downarrow_q \alpha_1 \qquad \langle \tau, \psi_2 \rangle \Downarrow_q \alpha_2}{\langle \tau, \psi_1 \oplus_a \psi_2 \rangle \Downarrow_q \alpha_1 \oplus_a \alpha_2} \qquad \frac{\langle \tau, a \rangle \Downarrow_a n}{\langle \tau, \chi[a] \rangle \Downarrow_q \chi[n]} \qquad \frac{}{\langle \tau, \alpha \rangle \Downarrow_q \alpha}$$

QWhile language – trace-based semantics

- Query command evaluation

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \qquad \texttt{query}(\alpha) = v \qquad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \texttt{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\texttt{skip}]^l, \tau :: \epsilon \rangle} \textbf{ query}$$

- 2. Compute query request and get the result: $\text{query}(\alpha) = v$

Expected value of the function $\lambda\chi\ .\alpha$ applied to each row of the hidden database:

$$\texttt{query}(\chi[3] + 5) = \frac{1}{n} \sum_{i=0}^{n} \chi_i[3] + 5$$

Summarize the computation procedure into $\text{query}(\alpha) = v$

QWhile language – trace-based semantics

- Query command evaluation

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \qquad \texttt{query}(\alpha) = v \qquad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \texttt{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\texttt{skip}]^l, \tau :: \epsilon \rangle} \textbf{ query}$$

- 3. Generate the event for query assignment : $\epsilon = (x, l, v, \alpha)$

x: the variable name

l: the label for this query command

v: the query result computed over unknown database

α: the normalized the query request

Adaptivity Formalization

- ➔ "***Reachability***" aspect:

  Data dependency relation
- ➔ "***Quantitative***" aspect:

  Times that each command is evaluated
- ➔ "***Structure***" carrying the two sub-properties

  Semantics-based dependency graph
- ➔ "***Adaptivity***" as a ***quantitative-reachability*** hyper-property

➔ "*Reachability*" as **dependency relation**[1]

- ***Variable may-dependency:***

$$\mathrm{DEP}(x^i, y^j, c) \in \{true, false\}$$

```
twoRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do (
      [x ← query(X[j])]3;
      [j ← j - 1 ]4;
      [a ← a + x]5);
[l ← query(X[k] * a)]6
```

- $DEP(x^3, a^5, \mathrm{c}),\ DEP(a^5, l^6, \mathrm{c})$

[1] Patrick Cousot. 2019. Abstract Semantic Dependency. SAS'19

➔ "***Quantitative***" as the weight of a labeled variable[1,2]

- ***Weight of each labeled variable x^l in program c:***

$$w: \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$$

if $\rho(\tau_0, k) = 10$

```
twoRounds(k) :=
[a ← 0]0;                                  ──→ 1
[j ← k]1;                                  ──→
while [(j > 0)]2 do (
        [x ← query(X[j])]3;                ──→
        [j ← i - 1 ]4;                     ──→ 10
        [a ← a + x]5);                     ──→
[l ← query(X[k] * a)]6                     ──→ 1
```

[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017
[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

Example – ***quantitative*** as evaluation **times**

```
twoRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do (
      [x ← query(𝒳[j])]3;
      [j ← j - 1 ]4;
      [a ← a + x]5);
[l ← query(𝒳[k] ∗ a)]6
```

- for a^0: $w = \lambda \boldsymbol{\tau_0}.1$,
- for x^3: $w = \lambda \boldsymbol{\tau_0}.\max(0, \boldsymbol{\rho}(\boldsymbol{\tau_0}, \boldsymbol{k}))$

➔ New "***structure*** " as semantics-based dependency graph

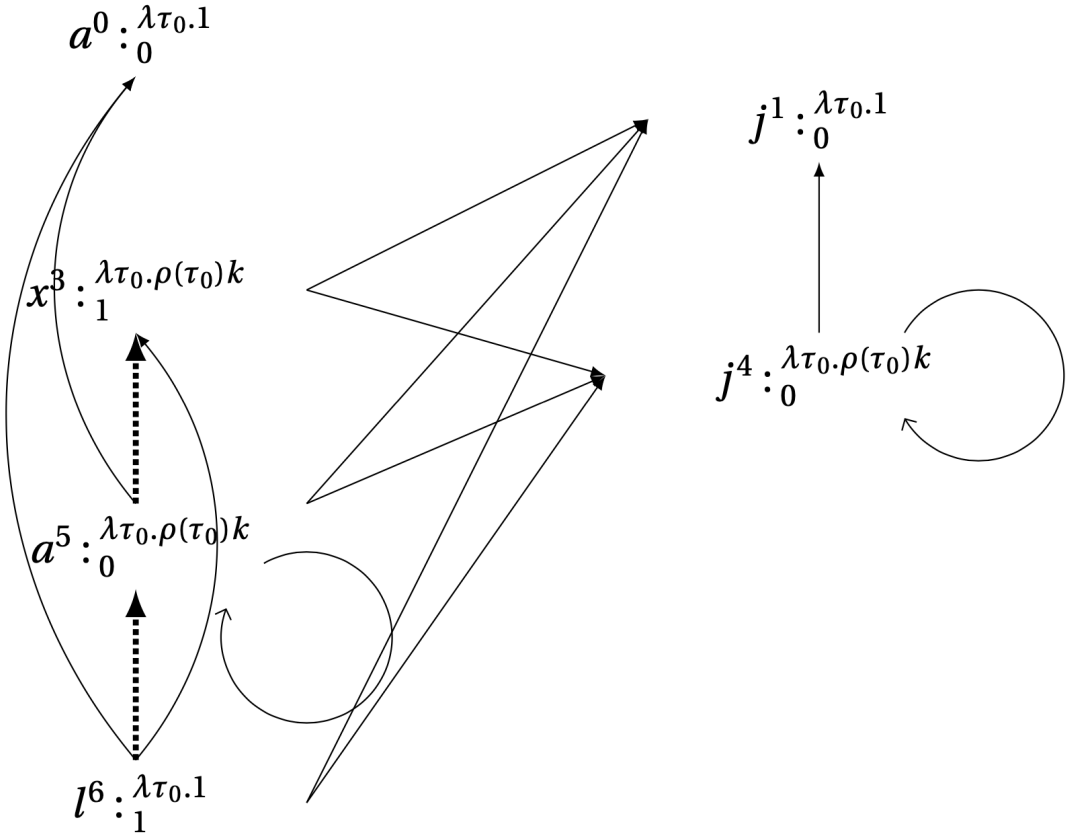
$$\boldsymbol{G(c) = (V(c), E(c), W(c), Q(c))}$$

- Vertex $\boldsymbol{V(c)}$: all the labeled variables in c;
- Edge $\boldsymbol{E(c)}$: ***variable may-dependency*** ;
- Weight $\boldsymbol{W(c)}$: ***weight*** of each labeled variable $\boldsymbol{x^l}$ ***in*** $\boldsymbol{c}$;
- Query Annotation $\boldsymbol{Q(c)}$: ***query request.***

carrying the two sub-properties

Example – "*structure* " as semantics-based dependency graph

```
twoRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do
(
[x ← query(X[j])]3;
[j ← j - 1 ]4;
[a ← a + x]5);
[l ← query(X[k] * a)]6
```



BOSTON UNIVERSITY

➔ ***Adaptivity*** as a "***quantitative-reachability***" hyper-property

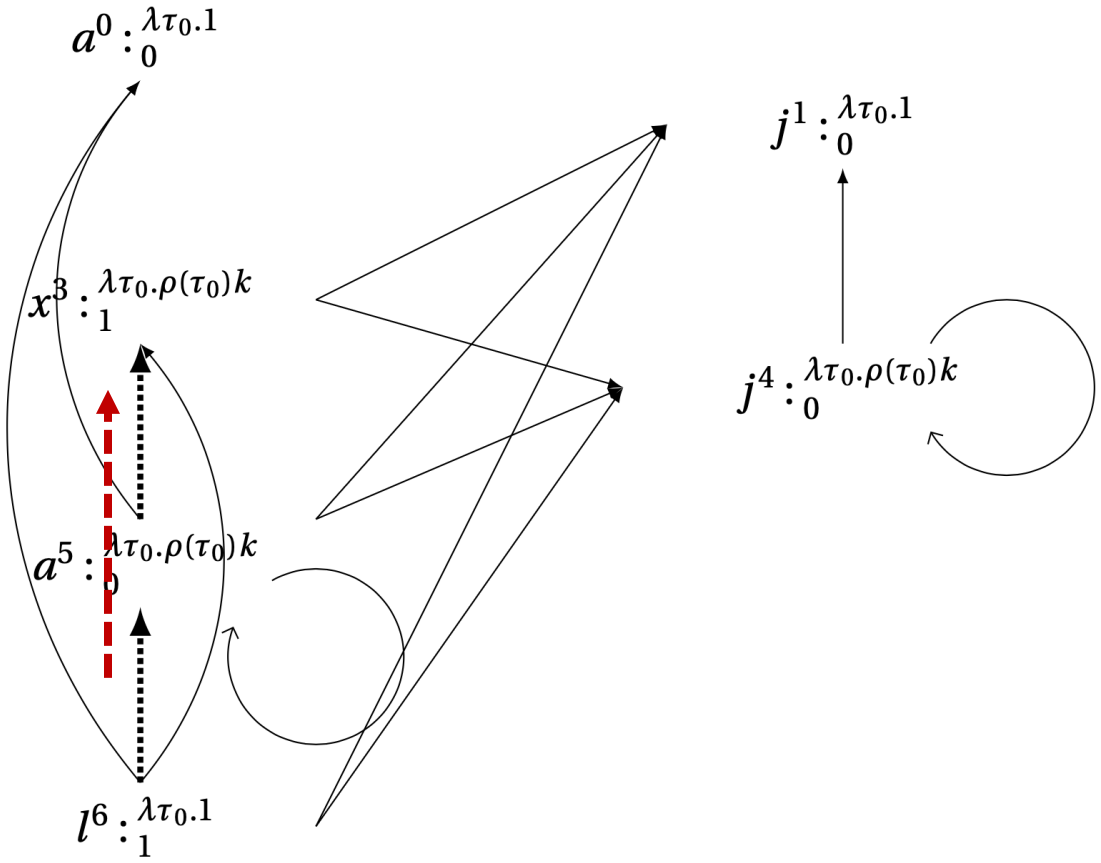
$$\textbf{\textit{Adaptivity}}\ \mathbf{A(c)} : \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$$

$$\mathbf{A(c)} = \lambda \tau_0 . \textbf{\textit{max}}\{\textbf{\textit{length}}\ \text{of the}\ \textbf{\textit{all the finite walks}}\ \}$$

- ***length*** : count only the query request
- ***finite walk*** : visiting times bounded by weight

Example – ***adaptivity*** as the ***length*** of the ***longest finite walk***

```
twoRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do
(
[x ← query(X[j])]3;
[j ← j - 1 ]4;
[a ← a + x]5);
[l ← query(X[k] * a)]6
```



BOSTON UNIVERSITY

Program Analysis Algorithm for Adaptivity

- ➔ "***Reachability***" sub-property:

data dependency estimation by data & control-flow analysis

- ➔ "***Quantitative***" sub-property :

weight estimation by reachability-bound analysis

- ➔ Estimate "***structure***"

estimated dependency graph

- ➔ "***Adaptivity***"

compute upper bound by longest walk estimation algorithm

➔ "***Reachability***": data-dependency analysis

data & control-flow and reaching definition analysis[1]

$$\text{flowsTo}(x^i, y^j, c) \in \{true, false\}$$

Soundness:

$$\forall c, x^i, y^j \in c \,.\, \text{DEP}(x^i, y^j, c) \Longrightarrow \text{flowsTo}(x^i, y^j, c)$$

[1] Nielson F., H.R. Nielson; , C. Hankin (2005). Principles of Program Analysis. Springer. ISBN 3-540-65410-0.

➔ "***Quantitative***" analysis[1]

$\hat{w} \in \boldsymbol{Expr}(\mathbb{N} \cup \boldsymbol{InputVar} \cup \{\infty\})$

```
twoRounds(k) :=
[a ← 0]0;                                  ──→ 1
[j ← k]1;                                  ──→
while [(j > 0)]2 do (
        [x ← query(X[j])]3;                ──→
        [j ← i - 1 ]4;                     ──→ k
        [a ← a + x]5);                     ──→
[l ← query(X[k] * a)]6                     ──→ 1
```

Soundness:

$$\forall c \in \mathscr{C}, x^l \in \mathrm{LV}(c), \tau_0 \in \mathscr{T}_0^+(c), \tau \in \mathscr{T}^+, v \in \mathbb{N}, l', (x^l, w) \in \mathtt{W_{trace}}(c) .$$

$$(x^l, \hat{w}) \in \mathtt{W_{est}}(c) \wedge \boxed{\langle \tau_0, \hat{w} \rangle \Downarrow_e v} \implies \boxed{w(\tau_0) \leq v}$$

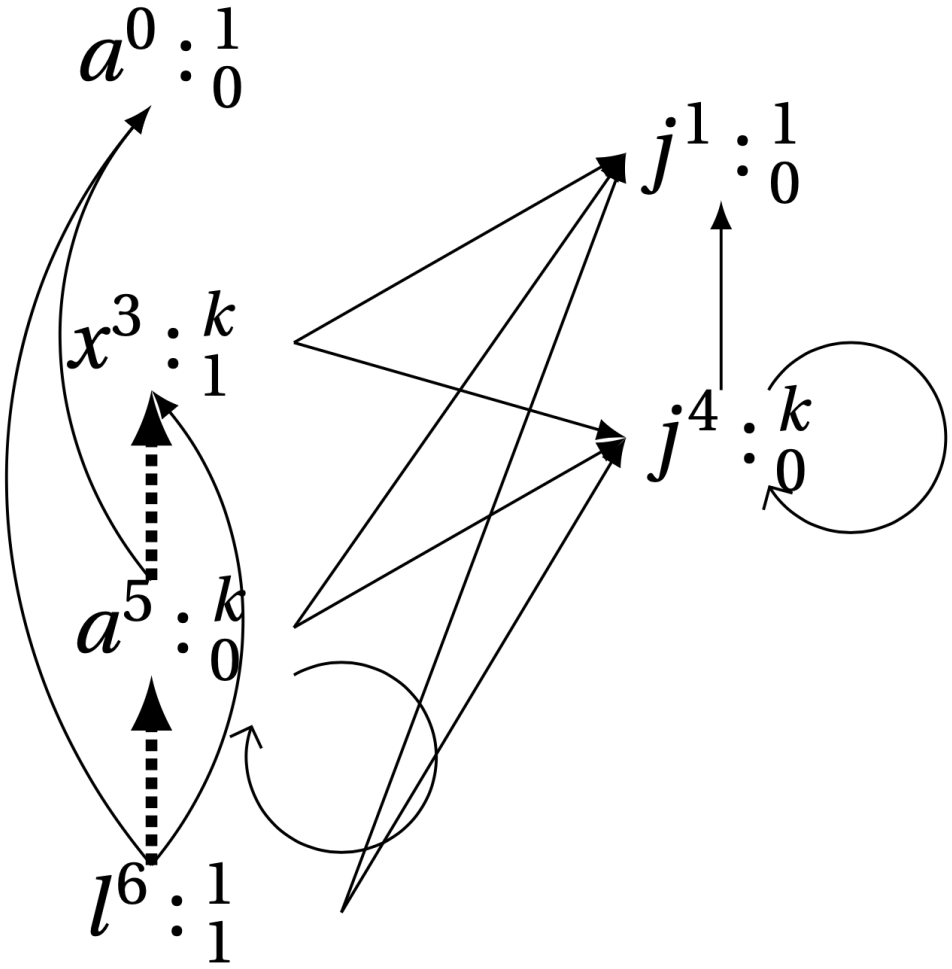
[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

➔ Estimate ***"semantics-based dependency graph"*** as $\widehat{G}(c)$

$$\widehat{\boldsymbol{G}}(\boldsymbol{c}) = (\boldsymbol{V}(\boldsymbol{c}), \widehat{\boldsymbol{E}}(\boldsymbol{c}), \widehat{\boldsymbol{W}}(\boldsymbol{c}), \boldsymbol{Q}(\boldsymbol{c}))$$

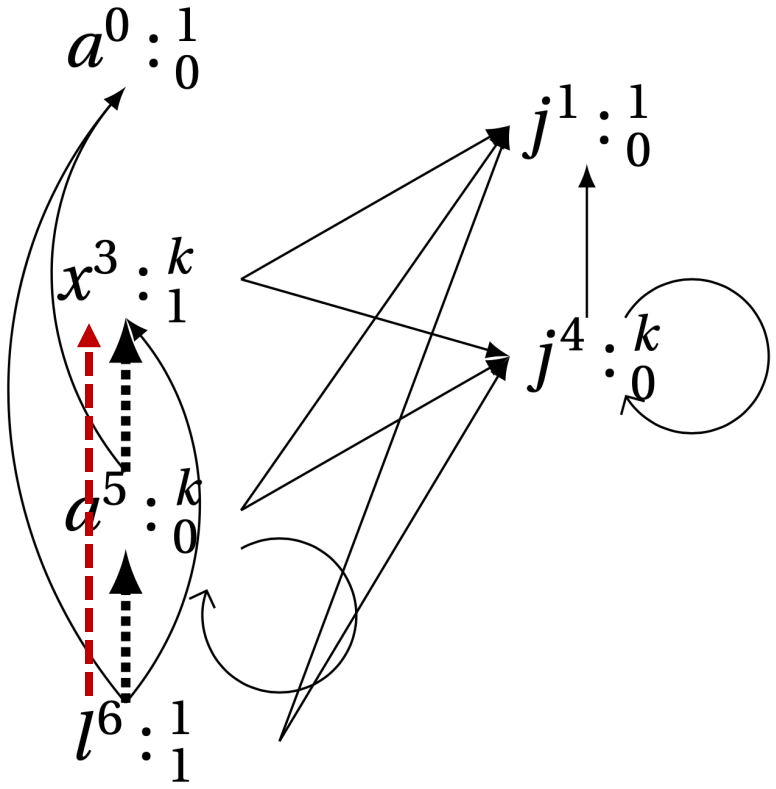
- Vertex $\boldsymbol{V}(\boldsymbol{c})$: labeled variables in c
- Edge $\widehat{\boldsymbol{E}}(\boldsymbol{c})$: ***Estimated data-dependency relation,*** $\text{flowsTo}(\boldsymbol{x^i}, \boldsymbol{y^j}, \boldsymbol{c})$
- Weight $\widehat{\boldsymbol{W}}(\boldsymbol{c})$: ***Estimated weight*** for each labeled variable $\boldsymbol{x^l}$ ***in*** $\boldsymbol{c}$;
- Query Annotation $\boldsymbol{Q}(\boldsymbol{c})$: labeled variables assigned by ***query requests***.

➔ *"Estimated dependency graph"*



➔ "***Adaptivity***" as the length of estimated longest walk

→ Estimate the upper bound on the program's "***adaptivity***"



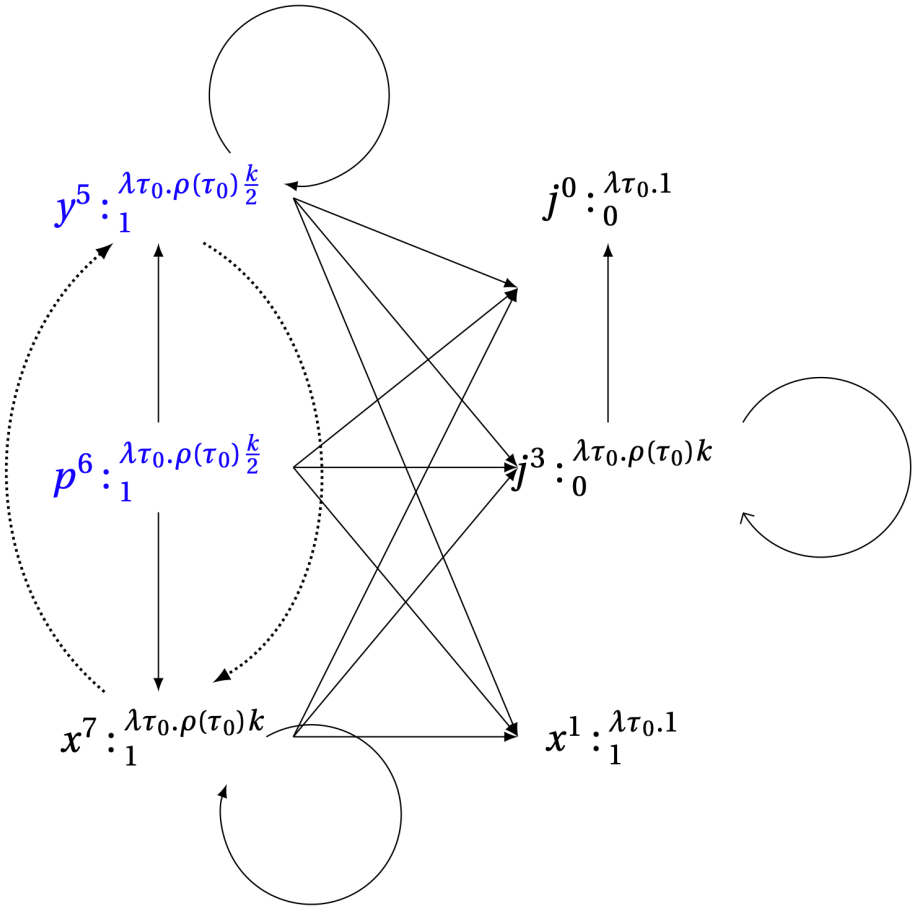
Limitation – imprecision of the ***program analysis*** algorithm

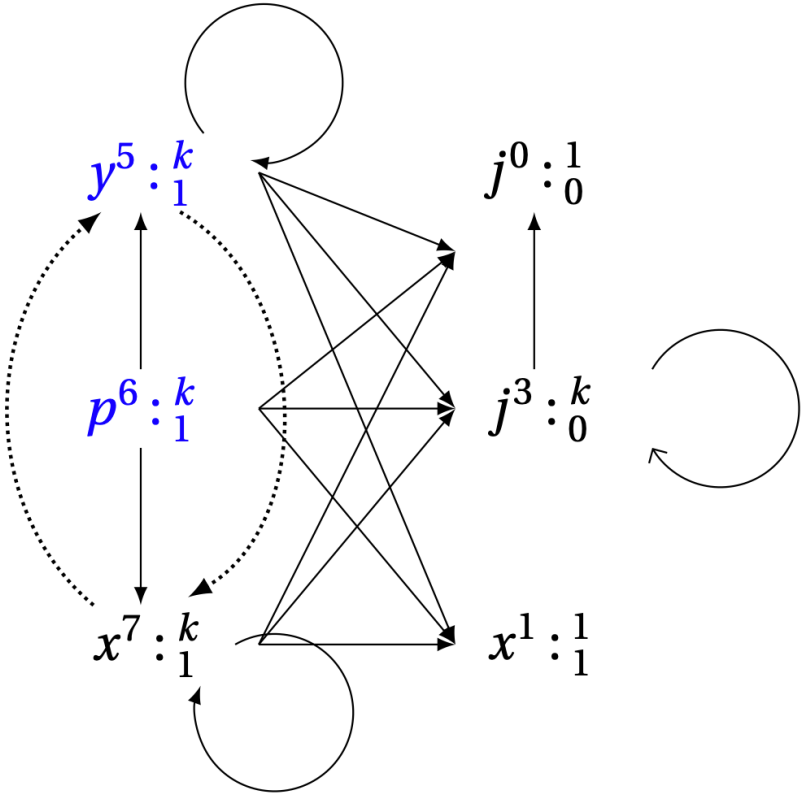
- The over-approximation in ***multiple paths*** loop bound

```
oddRounds(k) :=
[j ← 0]0;
[x ← query(𝒳[0])]1;
while [(j > 0)]2 do
(
    [j ← j - 1]3;
    if ([j%2 == 0]4,
        [y ← x]4;[p ← x]5),
        [x ← query(𝒳[y])] 6
        )
)
```

Limitation – imprecision of the ***program analysis*** algorithm

- The over-approximation in ***multiple paths*** loop bound



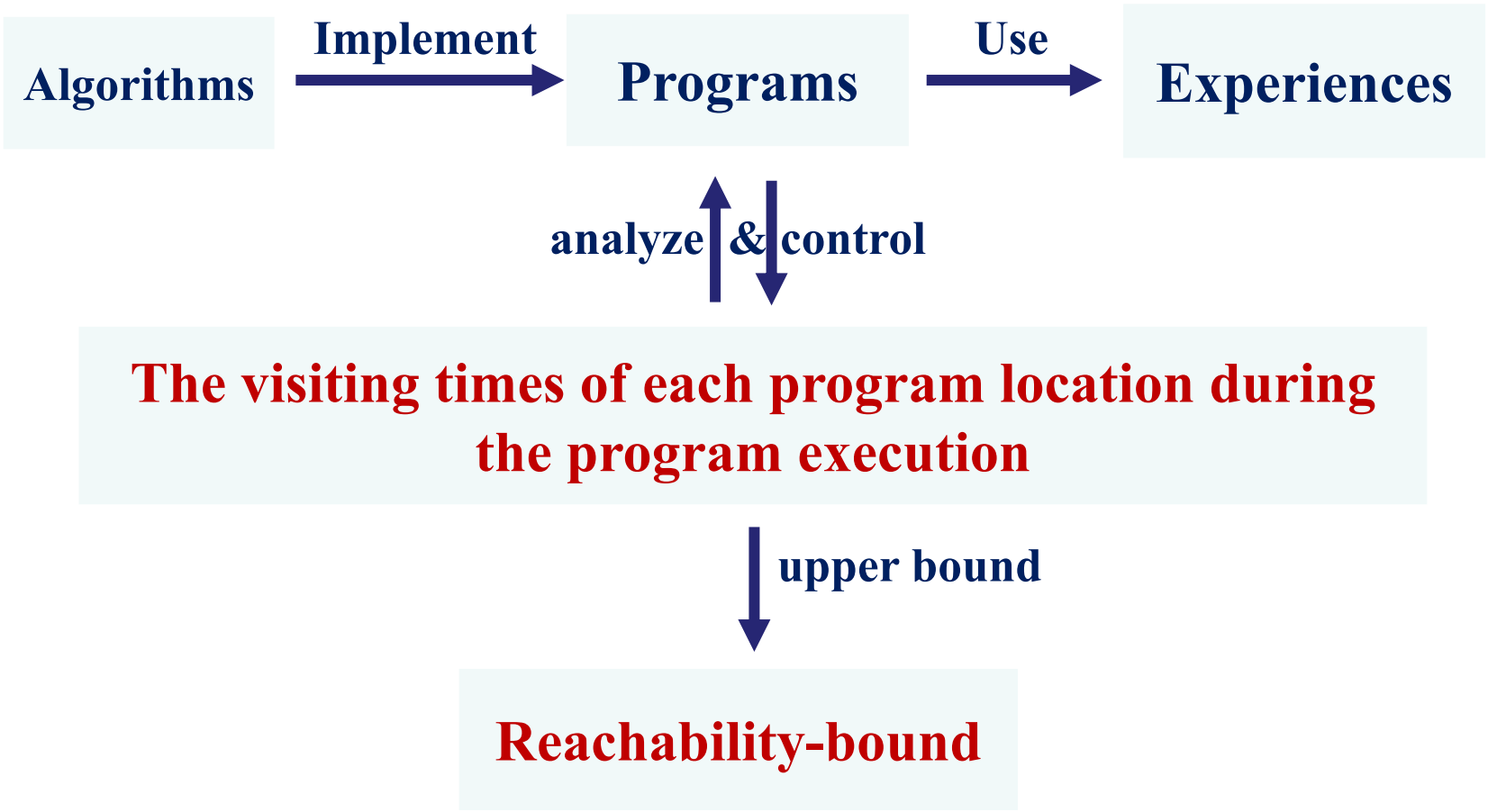


BOSTON UNIVERSITY

PART II :
Path-sensitive Reachability-bound Analysis

- Introduction:
  - Background
  - Motivation
- Language Model
- The Reachability-bound
- Path-sensitive Reachability-bound Algorithm

Introduction – background



Introduction – motivation

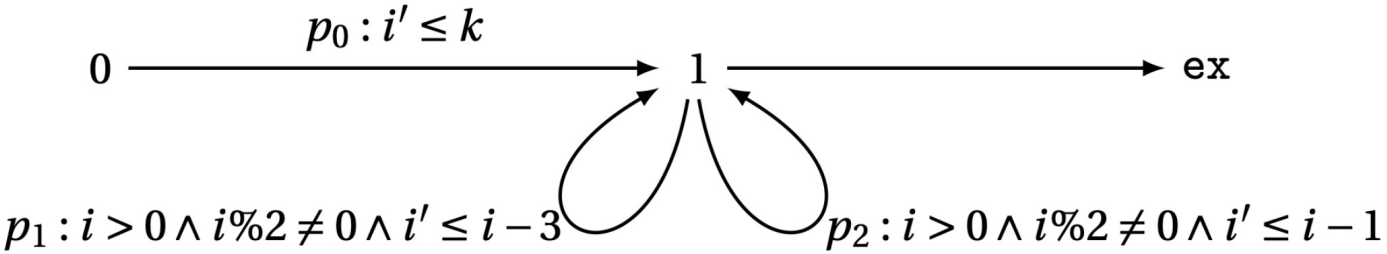
the number of times a given program location inside a procedure is visited during the program execution.

```
oddRounds(k) :=
[j ← 0]0;
[x ← query(𝒳[0])]1;
while [(j > 0)]2 do
(
[j ← j - 1]3;
if ([j%2 == 0]4,
 [y ← x]4;[p ← x]5),
 [x ← query(𝒳[y])] 6 )
)
```

simplify →

```
twoPaths(k) :=
[i ← k]^0;                    → 1
while [i > 0]^1 do            → k/2+1
(if([i%2==0]^2,               → k/2
then [i ← i - 1]^3,           → k/4
else [i ← i - 3]^4)           → k/4
```

Introduction – technique challenges in the path-sensitivity state-of-art[3]

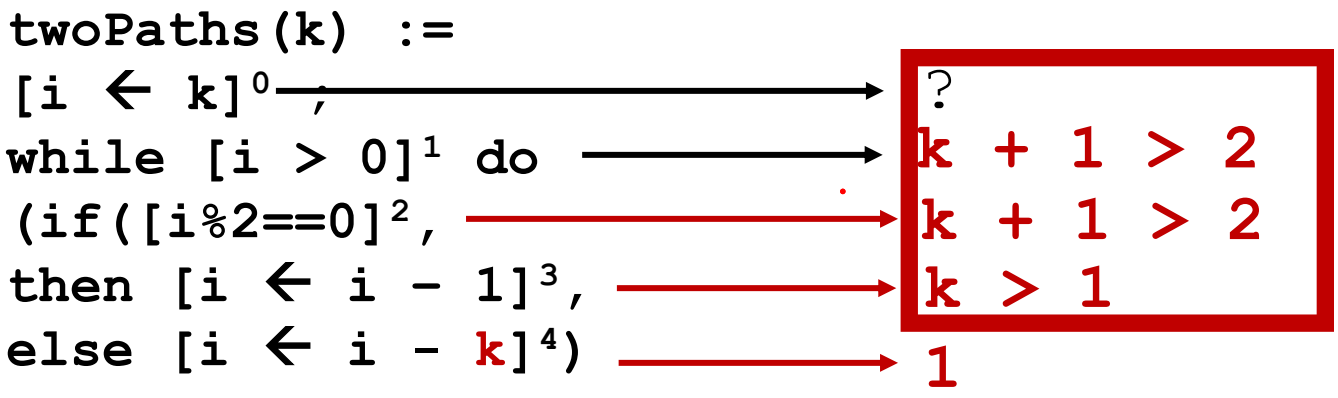



```
twoPaths(k) :=
[i ← k]⁰ ;                    ?
while [i > 0]¹ do             k/2
(if([i%2==0]², )              k/2
then [i ← i - 1]³,            k/2 > k/4
else [i ← i - 3]⁴)            k/2 > k/4
```

[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017
[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010
[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Introduction – technique challenges in the path-sensitivity

state-of-art[1, 2, 3]



[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017
[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010
[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

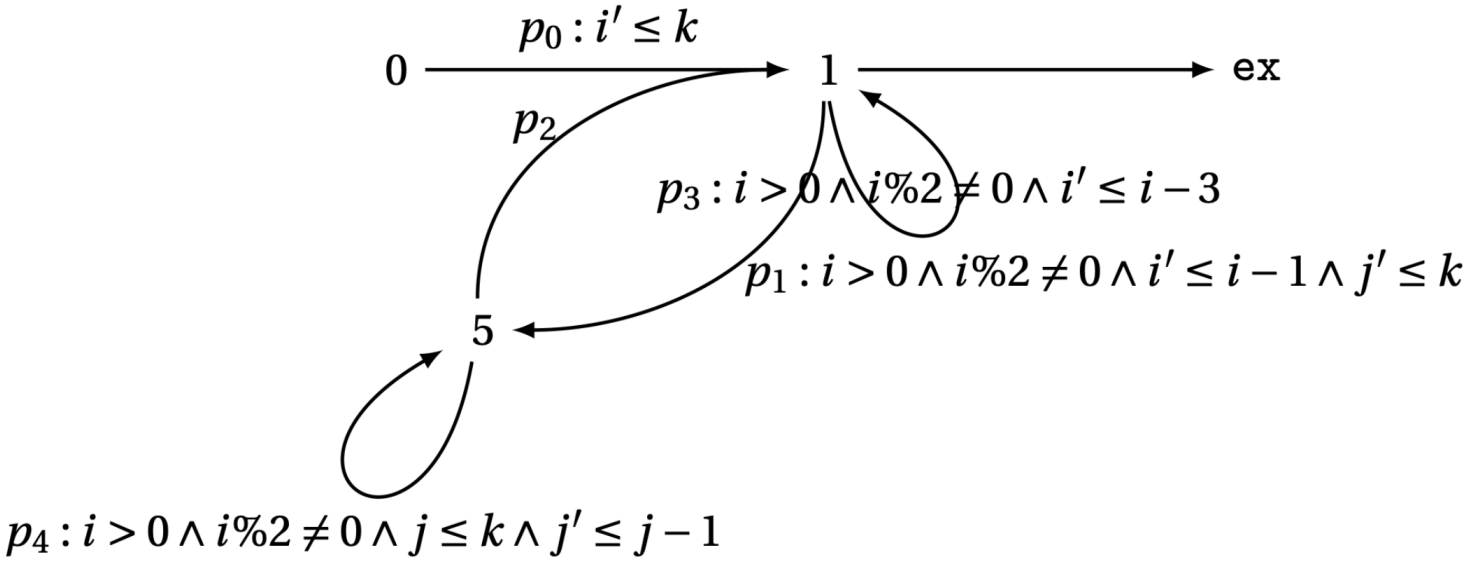
Introduction – technique challenges in nested loops

multiple loop paths and nested loops

```
loop2(n, m) :=
[i ← n]⁰ ;
while [i > 0]¹ do
(if([i%2==0]²,
   then
     [k ← i - m]³;  ⟶ m/4
     while [i > 0]⁴ do ([k ← i - m]⁵; ⟶ n-m
     [i ← k + m]⁶;
     [i ← i - 1]⁷,
   else
     [i ← i - 3]⁸
   )
```

Introduction – technique challenges in nested loops

state-of-art[1,2,3]



[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017
[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010
[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Introduction – technique challenges in nested loops

state-of-art[1,2,3]

```
loop2(n, m) :=
[i ← n]⁰ ;
while [i > 0]¹ do
(if([i%2==0]²,
   then
     [k ← i - m]³;
     while [i > 0]⁴ do ([k ← i - m]⁵;
     [i ← k + m]⁶;
     [i ← i - 1]⁷,
   else
     [i ← i - 3]⁸
   )
```

m/2

$$\frac{(n-m)*m}{2}$$

Introduction – main contribution

- New algorithm
  - compute reachability-bound on each location
  - locations on different path have different bounds
- New quantities:
  - Loop reachability-bound
  - Path reachability-bound
- Implementation and evaluations

[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017
[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010
[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Language model – expression evaluation

$$[\![]\!] : \mathcal{A} \cup \mathcal{B} \to \mathcal{T}^{+\infty} \to \bar{\mathcal{V}}$$

$[\![]\!]$: Arithmetic Expression → Trace → Arithmetic Value

$$\frac{}{[\![n]\!](\tau) = n} \qquad \frac{\rho(\tau)x = v}{[\![x]\!](\tau) = v} \qquad \frac{[\![a_1]\!](\tau) = v_1 \qquad [\![a_2]\!](\tau) = v_2 \qquad v_1 \oplus_a v_2 = v}{[\![a_1 \oplus_a a_2]\!](\tau) = v}$$

$$\frac{[\![a]\!](\tau) = v' \qquad \texttt{log}\ v' = v}{[\![\ \texttt{log}\ a]\!](\tau) = v} \qquad \frac{[\![a]\!](\tau) = v' \qquad \texttt{sign}\ v' = v}{[\![\ \texttt{sign}\ a]\!](\tau) = v}$$

The symbolic bound expression is a subset of the expressions, and we use the same rules and notation for evaluating the bound expression.

The reachability-bound – definition[1]

Definition 3 (Reachability-bound) *For a program c and a location $l \in \mathtt{L}(c)$ in c, a function $f(c,l) : \mathcal{T}_0^+(c) \to (\mathbb{N} \cup \{\infty\})$ is a* reachability-bound *for l if and only if*

$$\forall \tau_0 \in \mathcal{T}_0^+(c), c' \in \mathcal{C}, l, l' \in \mathtt{L}(c), \tau \in \mathcal{T}^{+\infty} \ .$$
$$\left(\langle c, \tau_0 \rangle \to^* \langle [\mathtt{skip}]^{l'}, \tau_0 +\!+ \tau \rangle \vee \langle c, \tau_0 \rangle \uparrow^\infty \tau_0 +\!+ \tau\right) \implies f(c,l)(\tau_0) \geq \mathtt{cnt}(\tau, l)$$

[1] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

Path-sensitive reachability-bound algorithm

1. Generate An Abstract Transition Graph
2. ***Refine*** the multipaths of loops in Program
3. Estimate the ***ranking functions*** and bound invariant
4. Estimate the ***path reachability-bound*** for every ***path***
5. Estimate the ***loop reachability-bound*** w.r.t. ***path*** and ***outer loops***
6. Estimate the ***reachability-bound*** for every ***program point***

➔ Abstract transition graph:

$$\texttt{absG}(c) \triangleq (\texttt{absV}(c), \texttt{absE}(c))$$

- ***Vertex***: $\texttt{absV}(c) = \texttt{L}(c) \cup \{\texttt{ex}\}$
- ***Edges***: $\hat{\epsilon} = (l, dc, l')$

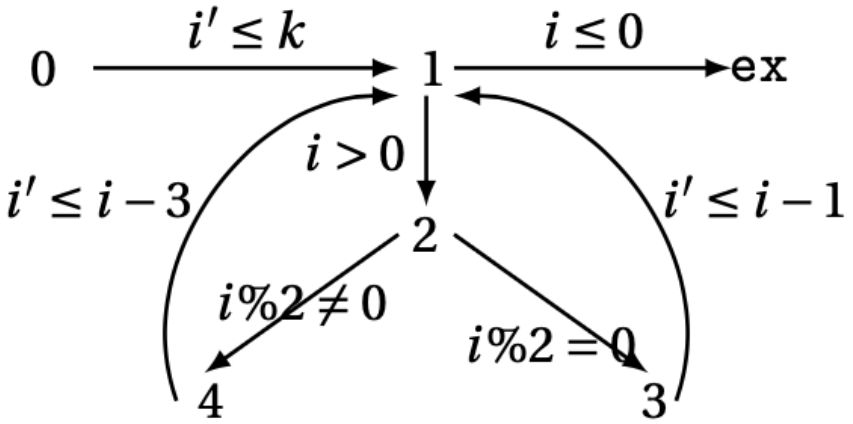
Constraints dc:

- Difference constraints
- Boolean expressions

contains every program point and transition

Example - abstract transition graph

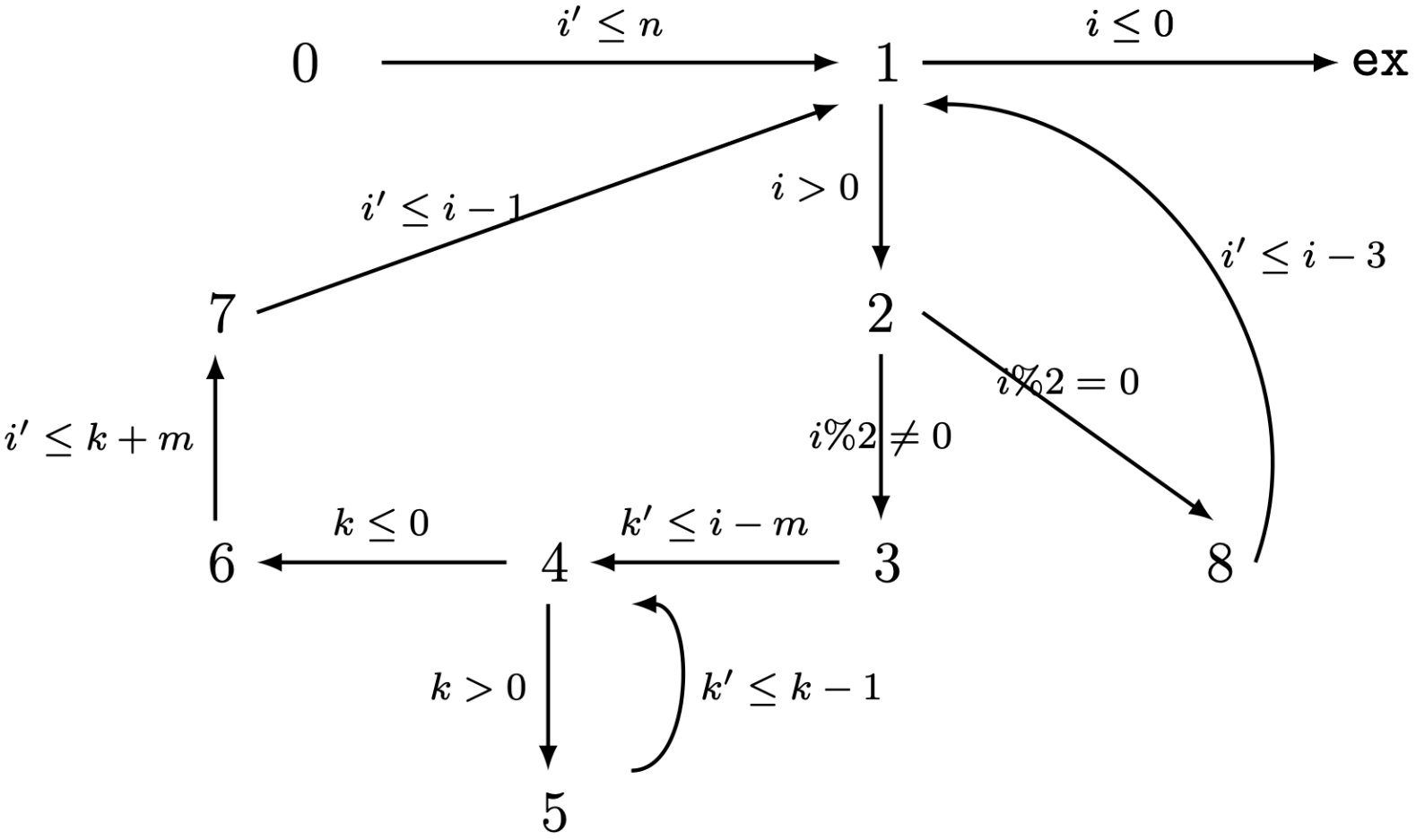
```
twoPaths(k) :=
[i ← k]⁰ ;
while [i > 0]¹ do
(if([i%2==0]²,
then [i ← i - 1]³,
else [i ← i - 3]⁴)
```



Example - abstract transition graph

```
loop2(n, m) :=
[i ← n]⁰ ;
while [i > 0]¹ do
(if([i%2==0]²,
   then
     [k ← i - m]³;
     while [i > 0]⁴ do ([k ← i - m]⁵;
     [i ← k + m]⁶;
     [i ← i - 1]⁷,
   else
     [i ← i - 3]⁸
   )
```

Example - abstract transition graph



➔ Abstract transition graph – Path

$$\mathtt{absG}(c) \triangleq (\mathtt{absV}(c), \mathtt{absE}(c))$$

- ***Path***:

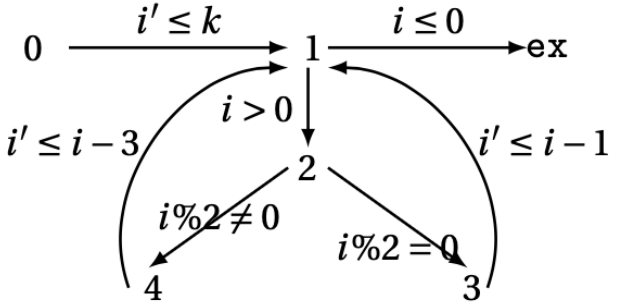
$$p \in \mathcal{PH}(absG(c))$$

  - path

$1 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow \text{ex}$

  - cyclic path

$1 \rightarrow 2 \rightarrow 4 \rightarrow 1$



  - simple path

$0 \rightarrow 1,\ 1 \rightarrow \text{ex},\ 1 \rightarrow 2 \rightarrow 3 \rightarrow 1,\ 0 \rightarrow 1 \rightarrow \text{ex}$

➔ Abstract transition graph – ***Simple transition path***

$$\texttt{absG}(c) \triangleq (\texttt{absV}(c), \texttt{absE}(c))$$

- ***Simple transition path***:

$$\text{tp} \in \mathcal{PH}(absG(c))$$

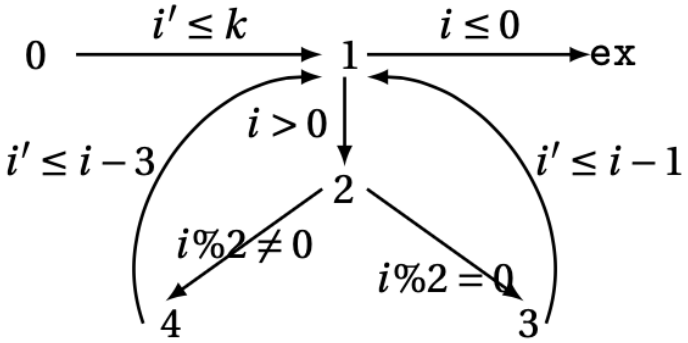
simple and start- or end- point **has to** be either

- a loop header
- or program entry
- or program exit.

and no loop header except the start- or end- points.

loop free and interleaving free

Example - simple transition path



- Simple transition path

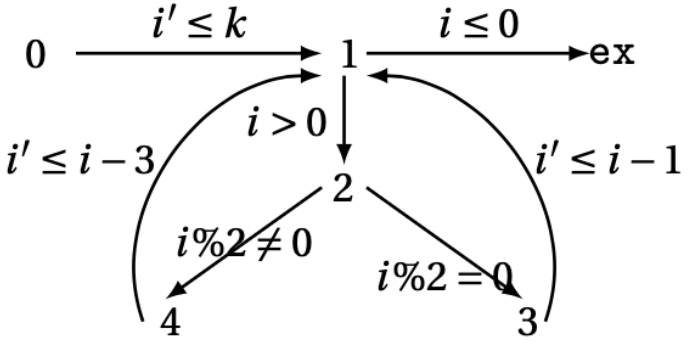
$0 \rightarrow 1,\ 1 \rightarrow \text{ex},\ 1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

- Path that is simple but not a simple transition path

$0 \rightarrow 1 \rightarrow \text{ex}$

➔ Program refinement – ***new loop paths***

- Rewrite and compute new loop paths

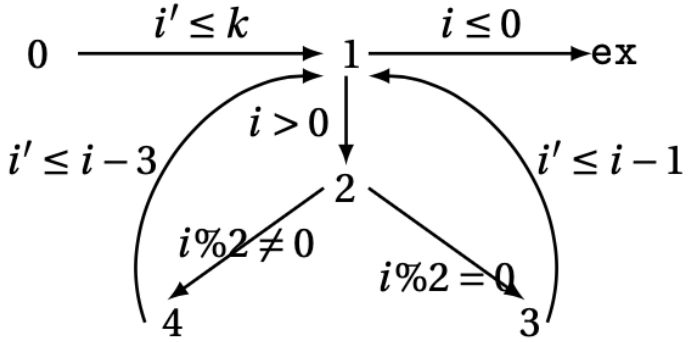


- New loop paths
  - rp1 = 1 → 2 → 3 → 1; 1 → 2 → 4 → 1 = tp1;tp2
  - rp2 = 1 → 2 → 4 → 1; 1 → 2 → 3 → 1 = tp2;tp1
- The refined program, ***rp***
  - rp = tp1; choose{repeat(rp1), repeat(rp2)}; tp3

Iteration times of ***new loop paths***.

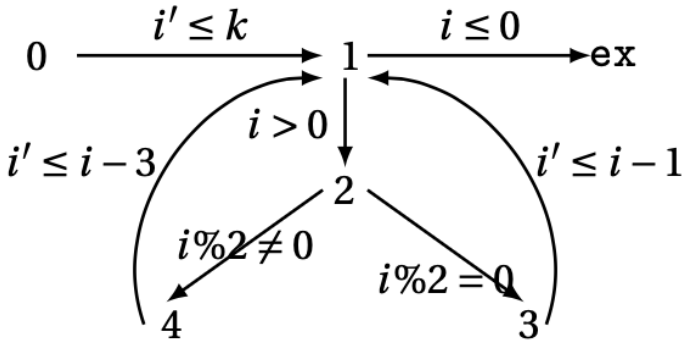
- $rp1 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 1; 1 \rightarrow 2 \rightarrow 4 \rightarrow 1$
- $rp2 = 1 \rightarrow 2 \rightarrow 4 \rightarrow 1; 1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

$$\frac{k}{4}$$



➔ Loop bound for ***new loop paths***

$$\textbf{\textit{BD(rp, c)}} \in Expr(\mathbb{N} \cup Input \cup \{\infty\}))$$

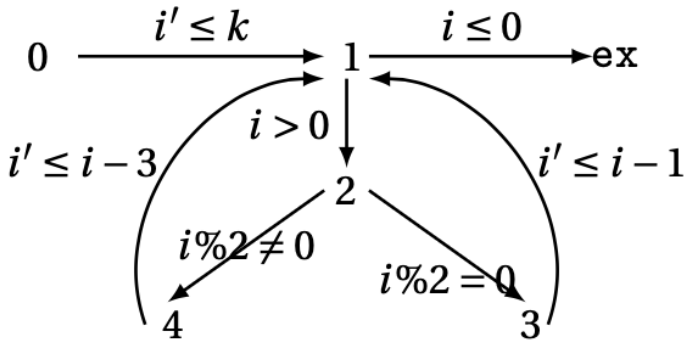


- $\text{BD}(1 \rightarrow 2 \rightarrow 3 \rightarrow 1; 1 \rightarrow 2 \rightarrow 4 \rightarrow 1) \Rightarrow \frac{k-0}{4}$
- $\text{BD}(1 \rightarrow 2 \rightarrow 4 \rightarrow 1; 1 \rightarrow 2 \rightarrow 3 \rightarrow 1) \Rightarrow \frac{k-0}{4}$

Iteration times of ***simple transition paths***.

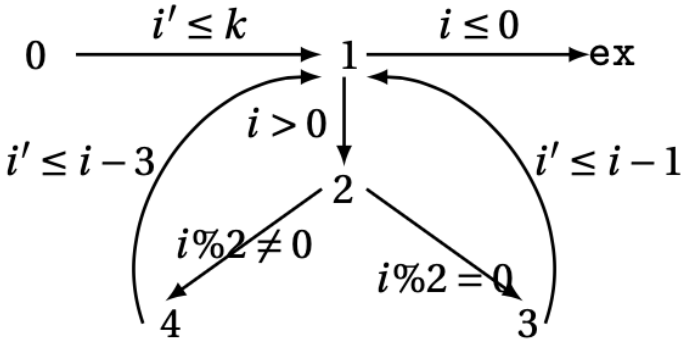
- tp1 = 1 → 2 → 3 → 1,
- tp2 = 1 → 2 → 4 → 1

$$\frac{k}{4}$$



➔ Path reachability-bounds – locally

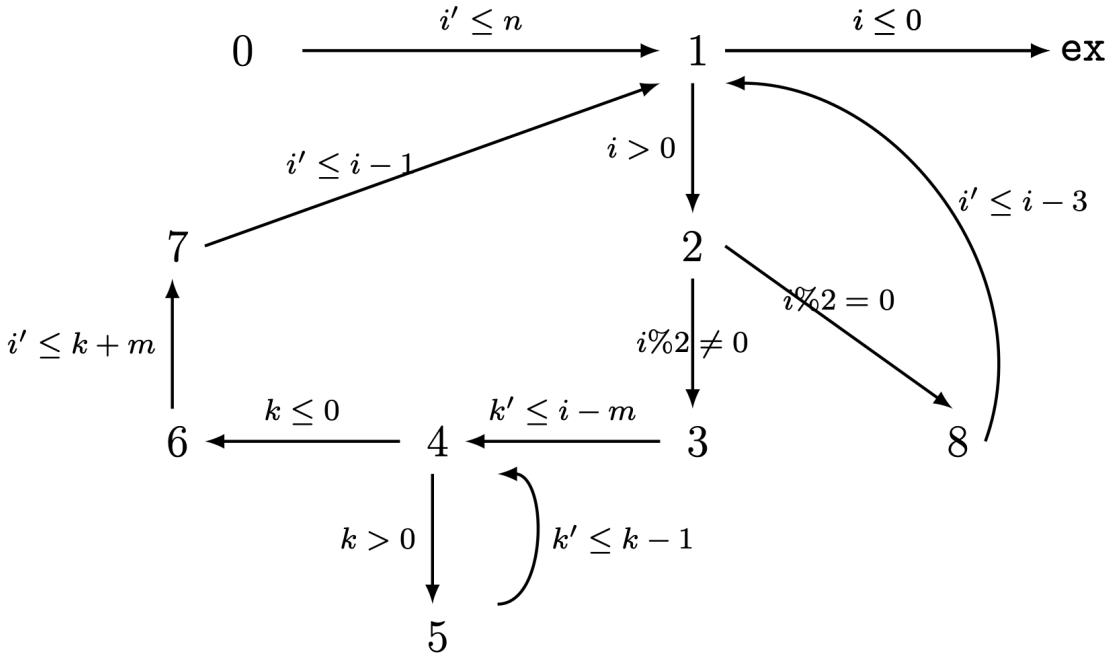
$$\text{pathLocRB}(\boldsymbol{rp}, \boldsymbol{tp}) \in \boldsymbol{Expr}(\mathbb{N} \cup \boldsymbol{Input} \cup \{\infty\}))$$



$$\begin{aligned}
&\text{pathlocRB}(\text{rp}, \text{tp1}) \\
&\quad = \max\begin{Bmatrix}\text{pathlocRB}(\text{rp1}, \text{tp1}), \\ \text{pathlocRB}(\text{rp2}, \text{tp1})\end{Bmatrix} \\
&\quad = \max\begin{Bmatrix}\text{pathlocRB}(\text{tp1}; \text{tp2}, \text{tp1}) * \text{BD}(\text{rp1}, \text{c}), \\ \text{pathlocRB}(\text{tp1}; \text{tp2}, \text{tp1}) * \text{BD}(\text{rp1}, \text{c})\end{Bmatrix} \\
&\quad = \max\left\{1 * \frac{\text{k}}{4}, 1 * \frac{\text{k}}{4}\right\} = \frac{\text{k}}{4}
\end{aligned}$$

Not precise in case with nested loops.

Example with nested loops

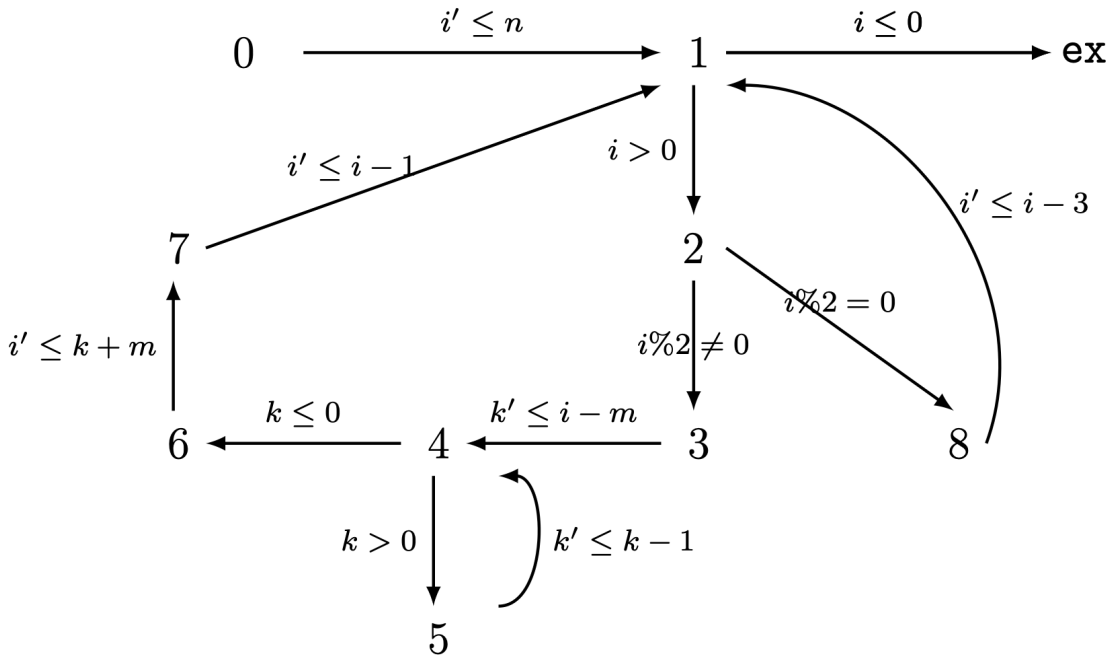


$$\text{pathlocRB}(\text{rp}, 4 \to 5 \to 4) = \frac{(n-m) * m}{4}$$

➔ Loop reachability-bounds

$$\text{lpRB}(\boldsymbol{rp'}, \boldsymbol{tp}) \in \boldsymbol{Expr}(\mathbb{N} \cup \boldsymbol{Input} \cup \{\infty\}))$$

The upper bound on iteration numbers of the outer loop, such that, during these iterations, the inner loop is entered.



$$\text{tp} = 5 \rightarrow 6 \rightarrow 5$$

outer loop:

$$\text{rp1} = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4; \text{repeat}(4 \rightarrow 5 \rightarrow 4); 4 \rightarrow 6 \rightarrow 7 \rightarrow 1$$

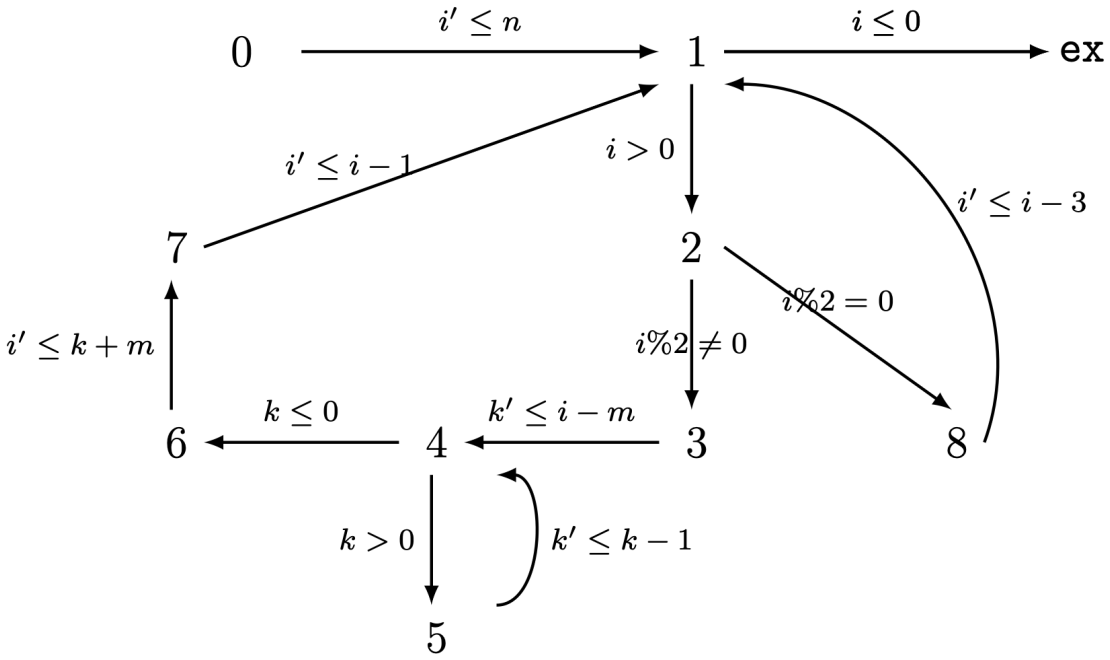
$$\text{loopRB}(\text{rp1}, \text{tp}) = 1$$

Path reachability-bound globally

$$\text{pathRB}(\boldsymbol{tp}, \boldsymbol{c}) \in \boldsymbol{Expr}(\mathbb{N} \cup \boldsymbol{Input} \cup \{\infty\}))$$

the upper bound on the execution times of a simple transition path, ***tp*** *when executing the program.*

loop2 example



$$\text{pathRB}(\text{rp}, \text{tp}) = \text{loopRB}(\text{rp1}, \text{tp}) * \text{pathlocRB}(\text{rp}, \text{tp})$$
$$= 1 * \text{pathlocRB}(\text{rp}, \text{tp2}) = \text{n} - \text{m}$$

7. Reachability-bound for every location

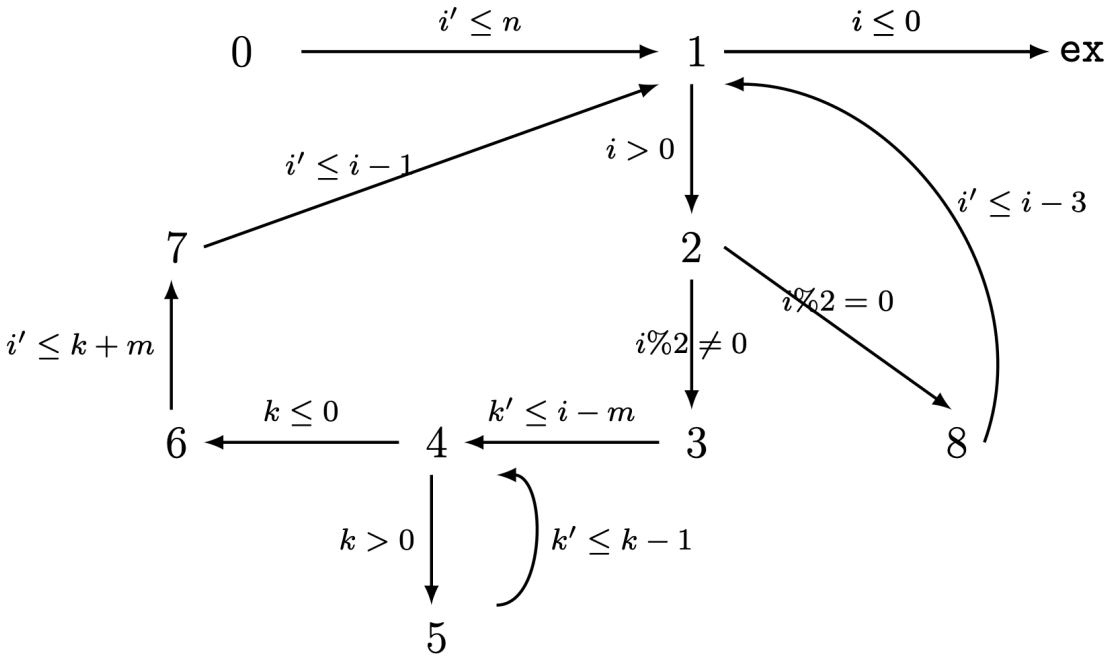
$$\mathrm{psRB}(\boldsymbol{tp}, \boldsymbol{c}) \in \boldsymbol{Expr}(\boldsymbol{N} \cup \boldsymbol{Input} \cup \{\infty\}))$$

$$\mathtt{psRB}(l, c) = \sum \{\mathtt{pathRB}(\mathtt{rp}, \mathtt{tp}, c) \mid \mathtt{tp} \in \mathtt{rp} \wedge l \in \mathrm{L}_{\mathtt{path}}(\mathtt{tp})\}$$

Soundness:

$[\![\mathtt{psRB}(l, c)]\!]$ is a reachability-bound for program c at location l

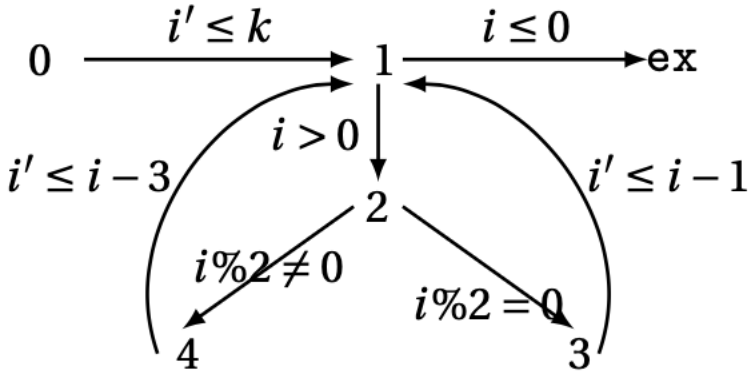
loop2 example



- $\text{psRB}(5) = \text{pathRB}(\text{rp}, 4 \rightarrow 5 \rightarrow 4\ ) = n - m$
- $\text{psRB}(2) = \text{pathRB}(\text{rp}, 1 \rightarrow 2 \rightarrow 3\ ) + \text{pathRB}(\text{rp}, 1 \rightarrow 2 \rightarrow 8 \rightarrow 1\ ) = \frac{m}{2}$

Example - reachability-bound

```
twoPaths(k) :=
[i ← k]⁰
while [i > 0]¹ do
(if([i%2==0]²,
then [i ← i - 1]³,
else [i ← i - 3]⁴)
```



- $\text{psRB}(4) = \text{pathRB}(\text{rp}, 1 \to 2 \to 4 \to 1) = \frac{m}{4}$
- $\text{psRB}(2) = \text{pathRB}(\text{rp}, 1 \to 2 \to 4 \to 1) + \text{pathRB}(\text{rp}, 1 \to 2 \to 3 \to 1) = \frac{m}{2}$

Implementation and empirical evaluation

Benchmarks: 270 from four benchmark sets

- 20 self-adopted programs
- 75.c programs from 7 benchmarks:

ABC: 15, C4B: 15, KoAT: 3, Loopus: 10, Rank10: 3, SPEED: 27, WTC: 37

- 37 challenging.c programs from the LoopusJAR [1]
- 37.java programs from Tianhan[2]
- 118.c programs from Icra [3]

[1] https://forsyte.at/static/people/sinn/loopusJAR/index.html
[2] https://zenodo.org/record/5140586#.Y5pBoC-B1QI
[3] https://github.com/icra-team/icra

Performance evaluation

Benchmark	# P	M.P.L #	Nested Loop #	Bounded	Success Rate	Failed	Time Outs	Total Runtime
Loopus	110	53	52	98	89.1%	11	7	7min42sec
Loopus-C	23	20	20	20	86%	2	3	12min39sec
Icra	118	111	21	107	90.1%	11	1	4min48sec
Tianhan	37	2	3	37	100.0%	1	0	1min03sec

Effectiveness evaluation

Table 13.1: The Effectiveness Evaluation Table of `psRB`

Bench--marks	theory worst	(#p)	(#l)	reachability-bound (#1)						worst bound (#p)				
				1	n	n^2	n^3	n^4	∞	psRB	Loopus	Cofloco	SPEED	Benchmark (2021a)
Other Tools	$O(1)$	4	82	32	0	0	0	0	0	1	2	3	2	1
	$O(n)$	59	373	112	261	0	0	0	0	48	51	45	46	40
	$O(n^2)$	31	414	70	191	153	0	0	0	37	29	34	37	49
	$n\log(n)$	6	81	0	0	0	0	0	0	0	0	0	0	0
	$O(n^3)$	2	91	6	16	13	13	0	0	4	3	2	5	7
	$O(n^4)$	3	36	4	9	9	12	11	0	4	4	3	5	5
Loopus	$O(1)$	1	154	0	0	0	0	0	0	0	1	0	0	0
	$O(n)$	13	176	49	105	0	0	0	0	12	13	14	14	11
	$O(n^2)$	4	64	4	13	15	0	0	0	3	2	5	2	6
	$O(n^3)$	4	193	2	7	10	4	0	0	2	1	2	2	3
	$O(n^4)$	1	21	1	3	6	8	2	0	1	1	1	1	0
Icra	$O(1)$	4	93	41	0	0	0	0	0	1	3	2	2	0
	$\log(n)$	2	16	0	0	0	0	0	0	0	0	0	0	0
	$O(n)$	79	988	267	698	0	0	0	0	78	80	82	78	77
	$O(n^2)$	12	151	26	110	26	0	0	0	18	14	11	16	17
	$O(n^3)$	1	9	1	3	3	2	0	0	1	1	4	2	4
	∞	20	295	50	0	0	0	0	245	9	0	0	0	0
Tianhan	$O(n)$	35	188	68	120	0	0	0	0	35	35	35	35	35
	$O(n^2)$	2	12	4	8	4	0	0	0	2	2	2	2	2

Accuracy evaluation

Bench--marks	theory worst	(#p)	(#l)	overall complexity bound	
				psRB	worst
Other Tools	$O(1)$	4	82	32	64
	$O(n)$	59	373	$112+261n$	$373n$
	$O(n^2)$	31	414	$70+191n+153n^2$	$414n^2$
	$n\log(n)$	6	81	0	0
	$O(n^3)$	2	91	$6+16n+13n^2+13n^3$	$48n^3$
	$O(n^4)$	3	36	$4+9n+9n^2+12n^3+11n^4$	$45n^4$
Loopus	$O(1)$	1	154	0	-
	$O(n)$	13	176	$49+105n$	$162n$
	$O(n^2)$	4	64	$4+13n+15n^2$	$32n^2$
	$O(n^3)$	4	193	$2+7n+10n^2+4n^3$	$23n^3$
	$O(n^4)$	1	21	$1+3n+6n^2+8n^3+2n^4$	$20n^4$
Icra	$O(1)$	4	93	6	64
	$\log(n)$	2	16	0	0
	$O(n)$	79	988	$267+698n$	$965n$
	$O(n^2)$	12	151	$26+110n+26n^2$	$162n^2$
	$O(n^3)$	1	9	$1+3n+3n^2+2n^3$	$9n^3$
	∞	20	295	50	∞
Tianhan	$O(n)$	35	188	$68+120n$	$188n$
	$O(n^2)$	2	12	$4+8n+4n^2$	$16n^2$

Future Works

Accurate Adaptivity Analysis Framework

- Accurate adaptivity definition
- Accurate adaptivity estimation with "path-sensitive reachability-bound algorithm"

Efficient Reachability-bound Algorithm

- More scalable implementation
- Leverage the possible non-terminating in program refining and the ranking function estimating.